

“Job Portal with resume parsing and matching”

A Project Report Submitted in the partial fulfillment of the requirements for the award of the degree of

BACHELOR OF TECHNOLOGY

In

DEPARTMENT OF COMPUTER SCIENCE ENGINEERING

&

DEPARTMENT OF COMPUTER SCIENCE AND INFORMATION TECHNOLOGY

By

A V S N R AKHILA : 2520030022

MANASVI NILESH GHAG : 2520030335

Under the Esteemed Guidance of

Dr Spandana S

Assistant Professor

Department of Computer Science and Engineering



Koneru Lakshmaiah Education Foundation

(Deemed to be University estd. u/s. 3 of the UGC Act, 1956)

Off-Campus: Bachupally-Gandimaisamma Road, Bowrampet, Hyderabad, Telangana - 500 043.

Phone No: 7815926816, www.klh.edu.in

**K L (Deemed to be) University
DEPARTMENT OF COMPUTER SCIENCE ENGINEERING**

&

DEPARTMENT OF COMPUTER SCIENCE AND INFORMATION TECHNOLOGY



Declaration

The Project Report entitled **“Job Portal with resume parsing and matching”** is a record of Bonafide work of **A V S N R AKHILA - 2520030022, MANASVI NILESH GHAG - 2520030335** submitted in partial fulfillment for the award of B. Tech in Computer Science and Engineering (or) Computer Science and Information Technology to the K L University. The results embodied in this report have not been copied from any other departments/University/Institute.

A V S N R AKHILA – 2520030022

MANASVI NILESH GHAG – 2520030335

K L (Deemed to be) University

DEPARTMENT OF COMPUTER SCIENCE ENGINEERING

&

DEPARTMENT OF COMPUTER SCIENCE AND INFORMATION TECHNOLOGY



CERTIFICATE

This is certify that the mini project based report entitled **“Job Portal with resume parsing and matching”** is a bonafide work done and submitted by **A V S N R AKHILA - 2520030022**, **MANASVI NILESH GHAG - 2520030335** in partial fulfillment of the requirements for the award of the degree of **BACHELOR OF TECHNOLOGY** in Department of Computer Science Engineering, K L (Deemed to be University), during the academic year **2024-2025**.

Signature of the Guide

Signature of the Course Coordinator

Signature of the HOD

ACKNOWLEDGEMENT

The success in this project would not have been possible but for the timely help and guidance rendered by many people. Our wish to express my sincere thanks to all those who has assisted us in one way or the other for the completion of my project.

Our greatest appreciation to my Course Coordinator **Dr Yerragudipadu Subbarayudu**, and my guide **Dr Spandana S**, Department of Computer Science which cannot be expressed in words for his/her tremendous support, encouragement and guidance for this project.

We express our gratitude to **Dr. P Venkateshwara Roa (CSE)/Dr. Kaja Shareef (CSIT)**, Head of the *Department for Computer Science Engineering/ Department for Computer Science and Information Technology* for providing us with adequate facilities, ways and means by which we are able to complete this project-based Lab.

We thank all the members of teaching and non-teaching staff members, and also who have assisted me directly or indirectly for successful completion of this project. Finally, I sincerely thank my parents, friends and classmates for their kind help and cooperation during my work.

A V S N R AKHILA – 2520030022

MANASVI NILESH GHAG – 2520030335

Abstract

Recruitment on most online job portals still depends on keyword search for candidates and on manual resume screening for recruiters. A job seeker has to retype the contents of a resume into long forms, and a recruiter has to open every application to judge whether the applicant actually has the skills a role needs. HireHub is a full-stack web application developed to close this gap. It is a role-based job portal for three kinds of users, namely candidates, recruiters and administrators, that turns an uploaded resume into structured data and uses that data to rank job openings by how well a candidate's skills match each job's stated requirements.

When a candidate uploads a resume in PDF or DOCX format, the back end extracts its text (falling back to optical character recognition for scanned PDFs), sends the text to the Google Gemini API for structured extraction of skills, education, experience and certifications, and falls back to a built-in rule-based parser whenever the AI service is unavailable. The extracted skills are stored against the candidate's profile and are used to compute a match score for every open job, defined as the percentage of a job's required skills that the candidate possesses. The score is shown to the candidate as job recommendations and is stored on every application so that recruiters can compare applicants. Around this core, the system implements the full hiring workflow: job posting and editing, applications with a seven-state status pipeline, interview scheduling with a calendar view and structured feedback, offer letters for hired candidates, saved jobs, recruiter statistics and a role-aware home page.

The application is built with React 19 and Vite on the front end, Node.js and Express exposing a REST API on the back end, and MongoDB accessed through the Mongoose ODM as the database. Authentication uses JSON Web Tokens with bcrypt-hashed passwords, and every protected route is guarded by role-based middleware that also enforces ownership, so that recruiters can only act on their own job postings. The system was verified against an isolated in-memory database with 51 scripted API test cases and 17 automated user-interface test cases run in a real browser; all but one passed, and the single failure exposed a PDF parsing limitation that is documented in this report. The project demonstrates how a document-oriented database, a stateless REST API and an AI-assisted parsing pipeline can be combined to make hiring faster and more skill-focused for both job seekers and recruiters.

Index

S. No.	Chapters	Topics	Page.no
		Acknowledgement	
		Abstract	1
1	Introduction	1.1 Background of the Project 1.2 Problem Statement 1.3 Scope of the Project 1.4 Objectives of the Project 1.5 Importance of the Application 1.6 Target Users / Audience	4-7
2	System Requirements	2.1 Hardware Requirements 2.2 Software Requirements 2.3 Development Tools and Frameworks	8-10
3	Technology Stack	3.1 Front-End Design 3.2 Back-End Design 3.3 Database Design 3.4 Version Control 3.5 APIs or External Services	11-15
4	System Architecture	4.1 High-level Architecture Diagram 4.2 Description of Each Layer (Frontend, Backend, Database) 4.3 Deployment Architecture	16-18
5	Design	5.1 ER Diagram / Database Schema	19-23
6	Implementation	6.1 Module-wise Implementation 6.2 Front-end Logic (UI Rendering, State Management) 6.3 API Integration 6.4 Authentication and Authorisation	24-31
7	Features	7.1 List of Main Features 7.2 How Each Feature Works (User Perspective + Technical Description)	32-37
8	Testing	8.1 Postman Testing (Frontend/Backend) 8.2 Integration Testing 8.3 Tools Used for Testing 8.4 Test Cases and Results	38-45
9	Deployment	9.1 Steps to Deploy 9.2 Environment Configuration 9.3 Hosting of Frontend and Backend	46-48
10	Challenges & Limitations	10.1 Issues Faced During Development 10.2 Solutions Applied 10.3 Current Limitations or Known Bugs	49-52
11	Future Enhancements	11.1 Planned Features 11.2 Possible Integrations or Optimisations	53-54
12	Conclusion	12.1 Summary of the Project 12.2 What Was Achieved 12.3 Skills Learned During Development	55-56
13	References	- Official documentation, standards, papers and books used	57-58

S. No.	Chapters	Topics	Page.no
14	Appendices	- Screenshots of the app - Sample code snippets - Installation/setup instructions - User manual or guide - Geo Tag photos with guide - Review forms with guide signatures	59-68

CHAPTER -1 INTRODUCTION

1.1 Background of the Project

Hiring has moved almost entirely online. Job seekers discover openings on portals, company career pages and professional networks, and recruiters receive applications through the same channels. The volume of applications that a single opening attracts has grown with this shift, and the effort of handling them has moved onto both sides of the process. Candidates repeatedly enter the same details into application forms, and recruiters spend a large share of their time reading resumes to decide which applicants are worth a closer look.

Most conventional job portals treat a resume as an attachment. The portal stores the file, but it does not understand what is inside it, so search and filtering depend on the keywords that a candidate typed into a profile and on the words that appear in a job title. Two people with the same skills can be ranked very differently because one of them described the skills with different words, and a recruiter has no quick way to see how closely an applicant's background matches the requirements written in a posting. The result is a process in which relevance is judged by hand, one document at a time.

At the same time, the tools needed to automate the first stage of this judgement have become widely accessible. Text can be extracted reliably from PDF and Word documents, optical character recognition (OCR) can recover text from scanned pages, and large language models offered through public APIs can turn unstructured resume text into structured fields such as skills, education and work history. When resumes are stored as structured data rather than as opaque files, comparing a candidate with a job becomes a computation instead of a reading task.

HireHub was developed as a project for the course Database Systems Engineering and Distributed Backend Development to apply these ideas in a complete, working system. It is a web-based job portal in which candidates upload a resume once, the system extracts their skills automatically, and every open job is scored against those skills. Recruiters post jobs with a list of required skills, review applicants together with their match scores and parsed resumes, move applications through a defined pipeline, schedule interviews, record interview feedback and issue offer letters, all inside the same application.

From a database and back-end perspective, the project is an exercise in modelling a real workflow as a set of related collections, exposing it through a stateless REST API, and protecting it with authentication and role-based authorisation. The domain contains clear entities (users, jobs, resumes, applications, interviews, offer letters and chatbot conversations) with one-to-many, many-to-many and one-to-one relationships between them, which makes it a suitable case study for document-database design with MongoDB and Mongoose. The application layer adds file handling, an external AI service with retry and fallback behaviour, and access rules that depend both on a user's role and on ownership of the data being accessed.

The system is implemented as two independent applications kept in one repository: a React single-page application in the `job-portal-frontend` folder and a Node.js/Express API in the `job-portal-backend`

folder. The front end communicates with the back end only through JSON over HTTP, which keeps the two layers loosely coupled and allows each to be developed, run and deployed separately.

1.2 Problem Statement

Existing job portals do not measure how well a candidate fits a job. They rely on keyword search for discovery and on manual resume screening for selection, so the skills contained in a resume are never compared systematically with the skills a job requires. This creates the following specific problems:

1. **Resumes are not machine-readable inside the portal.** A resume is stored as a file, so its skills, education and experience cannot be searched, compared or reused. Candidates are asked to re-enter the same information into forms.
2. **Relevance is judged by hand.** Recruiters must open each application to decide whether the applicant has the required skills, which is slow and inconsistent when many applications arrive for one role.
3. **Candidates cannot see why a job suits them.** Listings are ordered by date or keyword relevance rather than by how many of the required skills the candidate actually has.
4. **The hiring workflow is fragmented.** Application status, interview arrangements, interview feedback and offer letters are often handled through e-mail and spreadsheets outside the portal, so neither side has a single, current view of where an application stands.

The problem addressed by HireHub is therefore to design and build a job portal that converts resumes into structured data automatically, scores every job against a candidate's extracted skills, and supports the complete path from application to offer letter within one role-based system, while keeping each user's data visible only to the people entitled to see it.

1.3 Scope of the Project

The scope of HireHub covers a web application for three user roles and the back-end services and database that support them. The following functions are within scope:

- **User management:** registration and login for candidates and recruiters (an administrator option is also offered on the registration form and accepted by the API), password hashing, token-based sessions and profile editing.
- **Resume processing:** upload of PDF and DOCX resumes up to 5 MB, text extraction, OCR for scanned PDFs, AI-based structured extraction with a rule-based fallback, and candidate correction of the extracted fields.
- **Job management:** creation, editing, opening and closing of job postings by recruiters, and public browsing and searching of open jobs.
- **Matching:** a skill-overlap match score for every open job, recommendations for candidates, and a stored score on every application for recruiters.

- **Application workflow:** applying, withdrawing, saved jobs, and recruiter-controlled status changes across the stages applied, under review, shortlisted, interview, rejected and hired.
- **Interviews:** scheduling (online, in-person or phone), rescheduling, cancellation, completion and structured feedback with 1–5 ratings.
- **Offer letters:** creation of an offer letter for a hired application, resending, viewing by the candidate and download as a text file.
- **Dashboards and analytics:** a role-aware home page for candidates and recruiters, recruiter statistics per job, and a platform analytics endpoint for administrators.
- **AI assistant (API):** a chatbot endpoint that answers career and recruiting questions using Gemini and stores conversation history.

The following items are outside the current scope: e-mail or SMS notifications, online payments, video interviewing, integration with external job boards, mobile applications, and a production deployment. The administrator dashboard in the user interface is presently a static overview that is not yet connected to the analytics endpoint, and the chatbot is available through the API but has no user-interface component; both are discussed in Chapter 10.

1.4 Objectives of the Project

The project was undertaken with the following objectives:

1. **Automate resume parsing.** Convert an uploaded resume into structured skills, education, experience and certifications without manual data entry, and keep the system working when the AI service is unavailable.
2. **Score every job by skill match.** Compute, for each open job, the percentage of its required skills that a candidate has, use the score to recommend jobs, and store it with each application.
3. **Support hiring from application to offer.** Provide a single workflow for applying, reviewing, shortlisting, interviewing, recording feedback and issuing offer letters, with the current status visible to both parties.
4. **Enforce secure, role-based access.** Authenticate users with signed tokens and hashed passwords, and restrict every operation by role and by ownership of the underlying data.
5. **Design a sound document database.** Model the hiring domain as related MongoDB collections with validation, references, enumerations and unique indexes that protect data integrity.
6. **Deliver a responsive, usable interface.** Present role-specific dashboards and pages that work on desktop and mobile screens.

1.5 Importance of the Application

The importance of HireHub lies in moving the first, most repetitive stage of screening from people to the system, while leaving decisions with the recruiter. Its value can be seen from several angles.

For candidates, a single upload replaces repeated form filling. The parsed skills are shown back to the candidate in a Resume Insights section, where they can be corrected, and the match score explains which openings suit them best. The Recommended tab on the jobs page and the AI Matches list on the home page turn the job list into a ranked list that is specific to the candidate.

For recruiters, every applicant arrives with a match score and a structured summary of the resume, so the applicants who meet most requirements are visible at once. Status changes and offer letters are handled on the recruiter's Applications page, interview scheduling and feedback on the Interviews page (reached directly from an applicant with the application pre-selected), and the recruiter's home page summarises active jobs, applicants, interviews and hires.

For data integrity and privacy, the back end enforces rules that a portal built on shared spreadsheets cannot: a candidate can apply to a job only once (a unique compound index on job and candidate), an offer letter can only be created for a hired application and only once per application, recruiters can only see and change applications and interviews for jobs they posted, and recruiter feedback from interviews is never returned to candidates.

As an academic case study, the application brings together the main topics of database systems and back-end development in one coherent product: schema design with references and constraints, aggregation for analytics, a REST API with consistent status codes, stateless authentication, file processing and integration with an external AI service with retries and graceful fallback.

1.6 Target Users / Audience

HireHub defines three user roles in its data model (`candidate`, `recruiter` and `admin`). Their needs and the parts of the system they use are summarised in Table 1.1.

Table 1.1: User roles of HireHub

User role	Who they are	What they use in HireHub
Candidate	Students, fresh graduates and experienced professionals looking for full-time roles, internships, part-time, contract or remote work.	Resume upload and Resume Insights, AI job matches, job search with filters, saved jobs, applications and their status, interview calendar, offer letters, profile.
Recruiter	Hiring managers and HR staff who post openings on behalf of a company.	Job posting and editing, applicant review with match scores and parsed resumes, status pipeline, interview scheduling and feedback, offer letters, hiring statistics, company profile.
Administrator	Platform operators responsible for overall monitoring.	Platform analytics endpoint (counts of candidates, recruiters, jobs, applications and interviews, and applications by status); access to all applications and interviews through the API.

The intended audience of the application is therefore twofold: individuals seeking employment who benefit from automatic matching, and organisations that want a lightweight, self-contained tool to manage hiring for their openings. Because the interface is a responsive web application, both groups can use it from a laptop or a phone without installing any software.

CHAPTER -2 SYSTEM REQUIREMENTS

This chapter specifies the hardware and software needed to develop, run and use HireHub. The requirements are derived from the technologies the project actually depends on, as declared in the `package.json` files of the front end and back end, and from the way the application is run locally during development. Where a figure is a recommendation rather than a hard limit imposed by a tool, it is stated as such.

2.1 Hardware Requirements

HireHub runs as three local processes during development: the Vite development server for the React front end, the Node.js process for the Express API, and a MongoDB database server (or a connection to a hosted MongoDB cluster). None of these is computationally heavy for a single developer, but the build tooling, the browser, the code editor and the OCR library together benefit from a reasonably modern machine.

Table 2.1: Hardware requirements for development

Component	Minimum (recommended for development)	Reason
Processor	64-bit dual-core CPU (quad-core recommended)	Runs Node.js, the Vite dev server, MongoDB and a browser at the same time; resume OCR is CPU-bound.
Memory (RAM)	8 GB (16 GB recommended)	Node.js processes, the browser and the editor run concurrently; Tesseract OCR loads a language model into memory.
Storage	Approximately 2 GB free space	Two <code>node_modules</code> folders, MongoDB data files, uploaded resumes, and OCR language data cached on first use (about 10 MB).
Network	Broadband Internet connection	Installing npm packages, calling the Gemini API, and downloading the OCR language data the first time a scanned PDF is processed.
Display	1366 × 768 or higher	The dashboards are designed for desktop widths but are also responsive for mobile screens.

On the server side, the back end has modest requirements because it is a single stateless Node.js process. Its main resource needs arise during resume uploads: text extraction from PDF and DOCX files is fast, but when a scanned PDF contains fewer than 100 characters of embedded text the server renders up to five pages to images and runs OCR on them, which is the most CPU- and memory-intensive operation in the system. Uploaded files are stored on the server's local disk in an `uploads` folder, so the host also needs persistent storage proportional to the number of candidates (each resume is limited to 5 MB and only the latest resume per candidate is kept).

For end users the requirements are minimal. HireHub is used through a web browser, so any desktop, laptop, tablet or smartphone capable of running a current version of Chrome, Edge, Firefox or Safari is sufficient. The client stores only a small authentication record in the browser's local storage.

2.2 Software Requirements

The software environment consists of the JavaScript runtime and package manager, the database, the browser, and the libraries listed in the two `package.json` files. Table 2.2 lists the runtime and system software, and Tables 2.3 and 2.4 list the application libraries with the versions declared in the project.

Table 2.2: Runtime and system software

Software	Version / Notes	Purpose
Operating system	Windows 10/11, macOS or Linux	Any platform supported by Node.js and MongoDB; the project was developed on Windows.
Node.js	18 or later (the backend README states 18+); the API tests in Chapter 8 were run on Node.js 24	Runs the Express API and the Vite build tooling. Native <code>fetch</code> , <code>FormData</code> and ES modules are used.
npm	Bundled with Node.js	Installs dependencies and runs the scripts <code>dev</code> , <code>start</code> , <code>build</code> , <code>preview</code> and <code>seed:jobs</code> .
MongoDB	A local MongoDB server (default URI <code>mongodb://127.0.0.1:27017/hirehub</code>) or a hosted MongoDB cluster	Stores all application data; only the <code>MONGODB_URI</code> value changes between local and hosted databases.
Web browser	Current Chrome, Edge, Firefox or Safari	Runs the React single-page application.
Google Gemini API key	Optional	Enables AI resume parsing and chatbot replies; without a key the system uses its rule-based parser and fallback chatbot replies.

Table 2.3: Front-end dependencies (job-portal-frontend/package.json)

Package	Declared version	Role in the front end
react, react-dom	^19.1.1	Component-based user interface and rendering.
react-router-dom	^7.8.2	Client-side routing and protected routes.
framer-motion	^12.23.12	Entrance and hover animations on dashboard cards and lists.
lucide-react	^0.468.0	Icon set used throughout the interface.
three, @react-three/fiber, @react-three/drei	^0.179.1, ^9.3.0, ^10.7.6	Declared as dependencies but not imported by the current source code (see Chapter 10).
vite, @vitejs/plugin-react	^7.1.3, ^5.0.2	Development server with hot reload and production bundler.

Table 2.4: Back-end dependencies (job-portal-backend/package.json)

Package	Declared version	Role in the back end
express	^4.21.1	HTTP server, routing and middleware.
mongoose	^8.8.3	Schemas, validation, references and queries for MongoDB.
jsonwebtoken	^9.0.2	Signing and verifying JSON Web Tokens.
bcryptjs	^2.4.3	Password hashing with a cost factor of 10.
multer	^1.4.5-lts.1	Multipart file upload, file-type filter and 5 MB size limit.
pdf-parse	^1.1.1	Text extraction from PDF resumes.
mammoth	^1.8.0	Text extraction from DOCX resumes.

Package	Declared version	Role in the back end
pdf-to-img, tesseract.js	^7.0.0, ^7.0.0	Rendering scanned PDF pages to images and running OCR on them.
@google/generative-ai	^0.21.0	Client for the Google Gemini API (resume extraction and chatbot).
cors, dotenv	^2.8.5, ^16.4.5	Cross-origin access for the front end; loading configuration from a <code>.env</code> file.
nodemon (dev)	^3.1.7	Automatic restart of the API during development (<code>npm run dev</code>).

2.3 Development Tools and Frameworks

The project was built with a small set of widely used tools. Their roles are summarised below.

- **Visual Studio Code** was used as the code editor for both applications, with its integrated terminal for running the development servers.
- **Git and GitHub** were used for version control and for sharing the code between team members (Section 3.4).
- **Vite** provides the front-end development server on port 5173 with hot module replacement, and produces the optimised production bundle with `npm run build`.
- **nodemon** restarts the Express server automatically when back-end files change (`npm run dev`).
- **npm scripts** standardise common tasks: `dev`, `build` and `preview` in the front end; `dev`, `start` and `seed:jobs` in the back end.
- **Seed script** (`scripts/seedJobs.js`, run with `npm run seed:jobs`) creates demonstration recruiter accounts and approximately 150 varied job postings so that matching and search can be exercised with realistic data. It is safe to re-run because it skips recruiters and jobs that already exist.
- **Browser developer tools** were used to inspect network requests between the React application and the API, and to test responsive layouts.
- **Microsoft PowerPoint** was used for the project poster, whose architecture and ER diagrams are reused in Chapters 4 and 5.

The main frameworks are React on the client and Express on the server. React structures the interface as reusable function components that hold their own state with hooks, while Express structures the API as routers and middleware functions. Mongoose sits between Express and MongoDB, giving the otherwise schema-less database a defined schema with types, required fields, enumerations, default values and indexes. Together these choices keep the entire stack in one language, JavaScript, which simplified development and allowed the same data shapes (for example the structure of a parsed resume) to be used on both sides of the API.

CHAPTER -3 TECHNOLOGY STACK

HireHub uses a JavaScript-based stack in which a React single-page application talks to a Node.js and Express REST API, and the API stores its data in MongoDB through Mongoose. This combination is often called the MERN stack. Using one language across the whole system meant that request and response bodies, validation rules and data shapes could be reasoned about in the same terms on both sides. This chapter describes how each part of the stack was used in the project and why it was chosen.

3.1 Front-End Design

The front end lives in the `job-portal-frontend` folder and is built with **React 19** and **Vite 7**. React was chosen because the interface consists of many repeating, data-driven elements (job cards, applicant rows, interview entries, status chips and statistics tiles) that map naturally onto reusable components, and because its hooks (`useState`, `useEffect`) make it straightforward to load data from the API and keep the screen in sync with it. Vite was chosen as the build tool for its fast development server with hot module replacement and its simple configuration; the project needs only the official React plugin.

The source is organised into a small number of files. `src/main.jsx` mounts the application inside a `BrowserRouter`. `src/App.jsx` contains the application's pages and shared components as function components, together with helper functions for formatting dates, salaries and status labels. `src/api.js` is the only module that talks to the back end: it exports one asynchronous function per API operation (for example `login`, `uploadResume`, `getMatches`, `applyToJob`, `scheduleInterview` and `createOffer`), each of which builds the request, attaches the bearer token and converts error responses into JavaScript errors with the server's message. `src/components/JobsMegaMenu.jsx` holds a navigation component, and `src/styles.css` contains all styling.

Routing is handled by React Router 7. Public routes include the home page (`/`), login (`/login`), registration (`/register`) and the About page (`/about`). Candidate routes are `/jobs`, `/jobs/:id`, `/applications`, `/interviews` and `/profile`; recruiter routes are `/recruiter/jobs`, `/recruiter/applications`, `/recruiter/interviews` and `/recruiter/profile`; and the administrator route is `/admin`. Every route other than the public ones is wrapped in a `ProtectedRoute` component that redirects unauthenticated visitors to the login page and sends users with the wrong role to their own dashboard. The home route is role-aware: logged-out visitors see the marketing landing page, while logged-in candidates and recruiters see their own overview.

Styling is written in plain CSS rather than a CSS framework. The stylesheet defines the colour palette, typography, layout grids and the dashboard components, and uses media queries so that the navigation collapses into a mobile menu and multi-column layouts stack vertically on narrow screens. **Framer Motion** adds short entrance animations and hover effects to cards and list items, and **lucide-react** supplies the icons used in navigation, buttons and statistics tiles.

State management is deliberately simple. Each page owns its data in local component state and loads it from the API when it mounts. The only state shared across the whole application is the authentication record (token, role, name and e-mail), which is kept in the browser's `localStorage` under the key `hirehub_auth` and read by helper functions whenever a page needs to know who is logged in. This avoids the need for a global state library in an application whose pages are largely independent. Section 6.2 describes this logic in detail.

3.2 Back-End Design

The back end lives in the `job-portal-backend` folder and is a **Node.js** application written with ES modules (`"type": "module"`) using the **Express 4** framework. Express was chosen because it is minimal and explicit: the request pipeline is a sequence of middleware functions, which made it easy to add authentication, role checks and file uploads only to the routes that need them. The code follows a layered structure in which each folder has a single responsibility, as shown in Table 3.1.

Table 3.1: Back-end project structure

Folder / file	Responsibility
<code>server.js</code>	Creates the Express app, enables CORS for the configured origins, parses JSON bodies, serves the <code>uploads</code> folder statically, mounts all routers under <code>/api</code> , returns 404 for unknown API routes, handles uncaught errors, connects to MongoDB and starts listening (default port 8000).
<code>config/db.js</code>	Connects to MongoDB using <code>MONGODB_URI</code> and logs connection events.
<code>routes/</code>	Ten router modules (<code>auth</code> , <code>jobs</code> , <code>candidate</code> , <code>matches</code> , <code>applications</code> , <code>interviews</code> , <code>admin</code> , <code>chatbot</code> , <code>recruiter</code> , <code>offers</code>) that map HTTP methods and paths to controller functions and attach middleware.
<code>controllers/</code>	Request handlers containing the business rules: validation, ownership checks, database operations and response formatting.
<code>models/</code>	Seven Mongoose schemas: <code>User</code> , <code>Job</code> , <code>Resume</code> , <code>Application</code> , <code>Interview</code> , <code>OfferLetter</code> and <code>Conversation</code> .
<code>middleware/</code>	<code>auth.js</code> (<code>requireAuth</code> , <code>requireRole</code>) and <code>upload.js</code> (Multer disk storage, file-type filter and size limit).
<code>utils/</code>	<code>resumeParser.js</code> (text extraction, OCR and rule-based parsing), <code>aiResumeParser.js</code> (Gemini extraction with chunking, retries and fallback) and <code>matchScore.js</code> (skill-overlap score).
<code>scripts/seedJobs.js</code>	One-time script that creates demonstration recruiters and job postings.

Each request passes through the same stages: Express matches the route, `requireAuth` verifies the JSON Web Token in the `Authorization` header and attaches the user's id, role and e-mail to the request, `requireRole` rejects users whose role is not allowed for that route, and the controller then applies finer rules (for example, that a recruiter may only update applications for jobs they posted) before reading or writing the database. All responses are JSON, and errors use conventional HTTP status codes: 400 for invalid input, 401 for missing or invalid credentials, 403 for insufficient permissions, 404 for missing resources, 409 for conflicts such as duplicate e-mail addresses or duplicate applications, and 500 for unexpected failures.

The back end also hosts the two compute-heavy services of the system. The resume pipeline extracts text with `pdf-parse` or `mammoth`, recovers text from scanned PDFs with `pdf-to-img` and `tesseract.js`, and structures

it with Gemini or the rule-based parser. The match-score function compares a candidate's skills with a job's required skills. Both are pure modules in `utils/`, so they can be replaced or improved without changing any route.

3.3 Database Design

HireHub stores its data in **MongoDB**, a document-oriented database, and accesses it through **Mongoose 8**. MongoDB suits the domain because several entities have naturally nested, variable-length structure: a parsed resume contains arrays of education and experience entries whose length differs from candidate to candidate, a conversation contains an ordered list of messages, and interview feedback is a small group of related ratings. These are stored as embedded sub-documents inside their parent document, so they are read and written together without joins.

Entities that have an independent life cycle or are shared between users are stored in separate collections and linked by references (`ObjectId` values with a `ref` to the target model). For example, an `Application` references the `Job`, the candidate `User` and the `Resume` it was submitted with, and an `Interview` references its `Application`, `Job`, candidate and the recruiter who scheduled it. Mongoose's `populate()` is used in the controllers to replace these references with the referenced documents' selected fields when building responses. The design therefore combines embedding for owned, bounded data with referencing for relationships between independent entities.

Mongoose schemas provide the validation that MongoDB itself does not enforce: required fields, string trimming and lower-casing (for e-mail addresses), numeric ranges (1 to 5 for interview ratings), enumerations (job types, application stages, interview modes and statuses, offer statuses, user roles) and default values. Data integrity rules that span documents are enforced with indexes: a unique index on `User.email`, a unique compound index on `Application(job, candidate)` that prevents duplicate applications, and a unique index on `OfferLetter.application` that allows at most one offer letter per application. Every schema enables automatic `createdAt` and `updatedAt` timestamps. The full schema is presented in Chapter 5.

The connection string is supplied through the `MONGODB_URI` environment variable, so the same code runs against a local MongoDB server or a hosted cluster without modification. The administrator analytics endpoint demonstrates MongoDB's aggregation framework by grouping applications by status with a `$group` stage, while the remaining statistics are computed with `countDocuments` queries run in parallel.

3.4 Version Control

The project is version-controlled with **Git** and hosted in a **GitHub** repository. All work is on the `main` branch, which tracks the `origin/main` branch on GitHub. The repository holds both applications under a `project-files` folder, alongside project deliverables such as the review presentation.

A `.gitignore` file keeps generated and sensitive files out of the repository: both `node_modules` folders, the back end's `.env` file (which holds the database URI, JWT secret and API key), uploaded resumes in `job-`

portal-backend/uploads/ (an empty `.gitkeep` file preserves the folder itself), operating-system files, log files and PowerPoint lock files. Excluding `.env` is particularly important, because it ensures that secrets are never committed; instead, the repository provides `.env.example` with the variable names and placeholder values that each developer copies and fills in locally.

The commit history records the main milestones of the repository's organisation, including the initial upload of the project files, the initial commit, a merge with the GitHub copy of `main`, the addition of the PowerPoint lock-file rule to `.gitignore`, and the restructuring of the code into the `project-files` folder. Using Git allowed team members to work on the code independently, review each other's changes and return to earlier versions when needed.

Repository: https://github.com/ManasviGhag/DBSE-DBD_Project_HireHub

3.5 APIs or External Services

HireHub depends on only one external network service, the **Google Gemini API**. All other capabilities, including PDF and DOCX text extraction and OCR, are provided by open-source libraries that run inside the back-end process.

Google Gemini API

The Gemini API is accessed through the official `@google/generative-ai` client library using the key in the `GEMINI_API_KEY` environment variable. It is used for two purposes:

1. **Structured resume extraction** (`utils/aiResumeParser.js`). The resume text is sent with an instruction prompt and a JSON response schema that requires four fields: `skills`, `education`, `experience` and `certifications`. Because the response MIME type is set to `application/json`, the model returns machine-readable data that is parsed directly. The model identifier configured in the code is `gemini-3.8-flash`. Resumes longer than 15,000 characters are split into chunks of about 12,000 characters at paragraph, line or sentence boundaries, processed one after another, and merged with duplicates removed.
2. **Career and recruiting assistant** (`controllers/chatbotController.js`). Messages are sent to the `gemini-2.5-flash` model with a role-specific system instruction (one for candidates, one for recruiters and administrators) and the previous messages of the conversation as chat history.

The integration was designed to fail gracefully. If no API key is configured, resume parsing uses the rule-based parser and the chatbot returns a clearly labelled fallback reply. If a Gemini call fails because of rate limiting (HTTP 429) or temporary overload (HTTP 503), the parser retries up to four times, waiting for the delay suggested by the API or using exponential back-off capped at 30 seconds; a daily-quota error is not retried. Any other failure also falls back to the rule-based parser, so a resume upload never fails merely because the AI service is unavailable.

Local libraries used in place of external services

- **pdf-parse** extracts the text layer of PDF resumes.
- **mammoth** extracts raw text from DOCX resumes.
- **pdf-to-img** and **tesseract.js** render scanned PDF pages to images and recognise their text. The English language data for Tesseract (about 10 MB) is downloaded once on first use and cached in the operating system's temporary folder, which is the only other network access the back end performs.
- **bcryptjs** and **jsonwebtoken** provide password hashing and token-based authentication without relying on an external identity provider.

HireHub does not use payment gateways, e-mail delivery services, cloud file storage or third-party authentication. The offer-letter "resend" action, for example, records a new sent time but does not send an e-mail. These are natural extensions and are listed in Chapter 11.

CHAPTER -4 SYSTEM ARCHITECTURE

4.1 High-level Architecture Diagram

HireHub follows a three-tier client-server architecture. The presentation tier is a React single-page application running in the user's browser; the application tier is a stateless Node.js and Express REST API; and the data tier is a MongoDB database accessed through Mongoose. Three supporting components inside the application tier carry out resume parsing, match scoring and the AI chatbot. Figure 4.1 shows these components and the way they communicate.

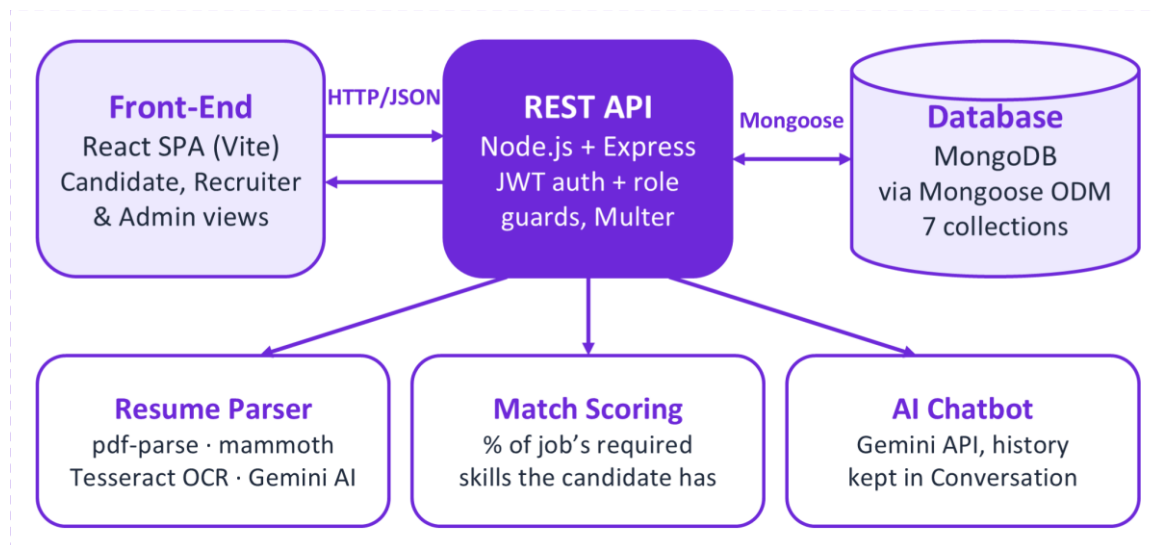


Figure 4.1: High-level architecture of HireHub

Communication between the tiers is strictly layered. The browser never connects to the database; it sends HTTP requests with JSON bodies (or multipart form data for resume uploads) to the API and receives JSON responses. The API is the only component that reads or writes MongoDB, and it is also the only component that calls the Gemini API, so the API key never reaches the browser. Because the API keeps no session state in memory (every request carries its own signed token), any number of API instances could serve the same front end and database.

A typical request, such as a candidate applying to a job, flows through the architecture as follows: (1) the React page calls `applyToJob(jobId, token)` in `api.js`; (2) the function sends `POST /api/jobs/:job_id/apply` with an `Authorization: Bearer` header; (3) Express routes the request through `requireAuth`, which verifies the token; (4) the controller checks that the user is a candidate, loads the job, rejects a duplicate application, loads the candidate's skills and latest resume, computes the match score and inserts the Application document; and (5) the API returns `201 Created` with the new application, which the page uses to update its state.

4.2 Description of Each Layer (Frontend, Backend, Database)

Frontend (presentation layer)

The frontend is responsible for everything the user sees and interacts with. It renders public pages (landing page, login, registration and About), the role-specific home pages, and the candidate and recruiter dashboards, each of which uses a shared `DashboardLayout` with a sidebar for navigation between Jobs, Applications, Interviews and Profile. It validates simple input in the browser (for example, required fields in the job-posting and offer-letter forms), formats data for display (relative times such as "6 days ago", salary ranges, interview modes and status labels), and keeps the user's authentication record in `localStorage`.

The frontend contains no business rules that the server relies on. It hides actions a user cannot perform (a recruiter does not see the Apply button, and the offer-letter panel appears only for hired applications), but every rule is enforced again by the API. The address of the API is read from the `VITE_API_URL` environment variable at build time and defaults to `http://localhost:8000/api`.

Backend (application layer)

The backend exposes the REST API and implements all business logic. Its internal layers are routes (URL and method mapping, middleware attachment), middleware (authentication, role checks and file upload), controllers (validation, ownership checks and orchestration), models (schemas and database access) and utilities (resume parsing and match scoring). The routes are mounted under the `/api` prefix: `/api/auth`, `/api/jobs`, `/api/candidate`, `/api/matches`, `/api/applications` (and `/api/jobs/:job_id/apply`), `/api/interviews`, `/api/offers`, `/api/recruiter`, `/api/admin` and `/api/chatbot`, plus a `/api/health` endpoint for monitoring. Uploaded resume files are stored on disk and served read-only under `/uploads` so that recruiters can open an applicant's original resume.

The backend also coordinates the slower operations. A resume upload performs text extraction, optional OCR, AI extraction or rule-based parsing, removal of the candidate's previous resume and its file, insertion of the new Resume document, and an update of the candidate's skills and years of experience, all within a single request handler.

Database (data layer)

The data layer is a MongoDB database with seven collections, one per Mongoose model: `users`, `jobs`, `resumes`, `applications`, `interviews`, `offerletters` and `conversations`. It stores account data (with passwords only as bcrypt hashes), job postings, parsed resumes, the application pipeline, interview records with feedback, offer letters and chatbot history. Referential links are stored as `ObjectIds` and resolved by the application with `populate()`, and integrity constraints are enforced by unique indexes and schema validation. The design of this layer is detailed in Chapter 5.

Table 4.1: Summary of architectural layers

Layer	Technology	Main responsibilities
Presentation	React 19, React Router 7, Vite 7, CSS, Framer Motion	Rendering pages and dashboards, routing and route protection, calling the API, storing the session token.
Application	Node.js, Express 4, JWT, bcrypt, Multer, Gemini client, pdf-parse, mammoth, Tesseract	REST endpoints, authentication and authorisation, validation, resume processing, match scoring, chatbot.
Data	MongoDB with Mongoose 8	Persistent storage of seven collections, schema validation, unique indexes, aggregation for analytics.

4.3 Deployment Architecture

HireHub has not been deployed to a public host. During development it runs entirely on a developer's machine, with the three processes shown in Table 4.2. The front end runs on the Vite development server, which proxies nothing and simply serves the React application; the browser then calls the API directly on its own port, which is permitted by the API's CORS configuration.

Table 4.2: Local development deployment

Process	Started with	Default address
React front end (Vite dev server)	<code>npm run dev</code> in <code>job-portal-frontend</code>	<code>http://localhost:5173</code>
Express API	<code>npm run dev</code> (nodemon) or <code>npm start</code> in <code>job-portal-backend</code>	<code>http://localhost:8000/api</code>
MongoDB	Local mongod service, or a hosted cluster referenced by <code>MONGODB_URI</code>	<code>mongodb://127.0.0.1:27017/hirehub</code>

Because the tiers communicate only over HTTP and are configured entirely through environment variables, the same code can be deployed to separate hosts without changes to the source. A deployment architecture for production is proposed in Chapter 9: the front end is built into static files with `npm run build` and served from a static hosting service or content delivery network; the API runs as a Node.js web service with its environment variables set in the host's configuration; and the database is a managed MongoDB cluster. Two aspects of the current design must be addressed in such a deployment: uploaded resumes are stored on the API server's local disk, which requires persistent storage (or a move to object storage), and `CORS_ORIGIN` must be set to the public address of the front end. The project does not currently include a continuous-integration pipeline, container files or infrastructure configuration.

CHAPTER -5 DESIGN

5.1 ER Diagram / Database Schema

The data model of HireHub was derived directly from the Mongoose schemas in `job-portal-backend/models/`. Figure 5.1 presents it as an Entity–Relationship diagram in Chen notation: rectangles are entities, ovals are attributes (the key attribute `_id` is underlined), diamonds are relationships, and the labels 1, N and M give the cardinality at each end of a relationship. For readability the diagram shows each entity's key and its most important attributes; the complete attribute lists follow in Tables 5.1 to 5.7.

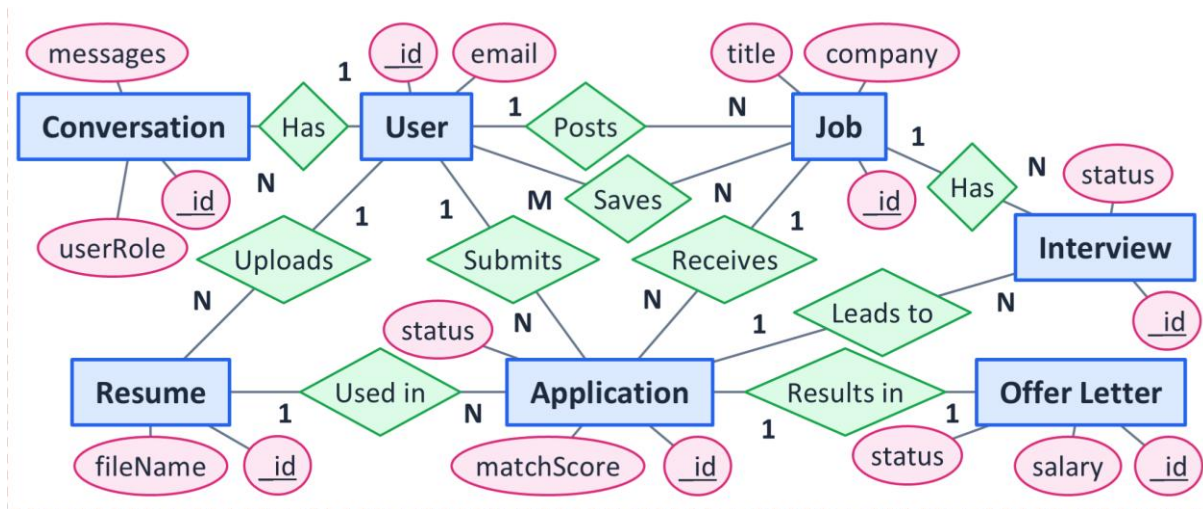


Figure 5.1: ER diagram of the HireHub database (Chen notation)

The relationships shown in the diagram correspond to references stored in the documents:

- **User Posts Job (1:N)** – `Job.postedBy` references the recruiter who created the job.
- **User Saves Job (M:N)** – `User.savedJobs` is an embedded array of { `job`, `savedAt` } entries, so one user can save many jobs and one job can be saved by many users.
- **User Uploads Resume (1:N)** – `Resume.candidate` references the candidate. The schema allows many resumes per user, although the upload logic keeps only the latest one.
- **User Submits Application (1:N)** – `Application.candidate` references the applicant.
- **User Has Conversation (1:N)** – `Conversation.user` references the user who chatted with the assistant.
- **Job Receives Application (1:N)** – `Application.job` references the job applied for.
- **Job Has Interview (1:N)** – `Interview.job` references the job the interview is for.
- **Resume Used in Application (1:N)** – `Application.resume` references the resume that was the candidate's latest when they applied (optional).
- **Application Leads to Interview (1:N)** – `Interview.application` references the application; one application may have several interview rounds.

- **Application Results in Offer Letter (1:1)** – `OfferLetter.application` references the application and is unique, so an application has at most one offer letter.

Some documents also hold denormalised references that duplicate a relationship already reachable through another entity, for faster and simpler queries: `Interview.candidate` and `Interview.scheduledBy`, and `OfferLetter.candidate`, `OfferLetter.job` and `OfferLetter.recruiter`. For example, storing the candidate on each interview lets the API fetch a candidate's interviews with a single query on `candidate` rather than first finding all of the candidate's applications. These shortcut references are listed in the tables but are omitted from the ER diagram because they do not represent separate relationships.

All seven schemas enable Mongoose timestamps, which add `createdAt` and `updatedAt` fields maintained automatically on every insert and update. Every document also has the primary key `_id` of type `ObjectId` generated by MongoDB. These three fields are not repeated in each table below. The notation "ref → X" means that the field stores the `ObjectId` of a document in collection X.

Table 5.1: users collection (model User)

Field	Type	Key / constraint	Description
name	String	Required, trimmed	Display name of the user.
email	String	Required, unique index , lower-cased, trimmed	Login identifier; duplicates are rejected with HTTP 409.
password	String	Required	bcrypt hash (cost 10); never returned in API responses.
role	String	Required, enum: candidate, recruiter, admin; default candidate	Determines which routes and pages the user can access.
company, industry, companyWebsite, companyDescription	String	Optional, trimmed	Recruiter company profile.
skills	[String]	—	Candidate skills copied from the latest parsed resume; used for match scoring.
experienceYears	Number	Default 0	Years of experience estimated from the resume text.
title, phone, location, linkedin, github, about	String	Optional, trimmed	Candidate profile details (phone and location are also used by recruiters).
savedJobs	[{ job, savedAt }]	job: ref → Job, required; savedAt: Date, default now	Bookmarked jobs (the M:N Saves relationship).

Table 5.2: jobs collection (model Job)

Field	Type	Key / constraint	Description
title	String	Required, trimmed	Job title, searchable with a case-insensitive pattern.
company	String	Required, trimmed	Hiring company.
location	String	Optional, trimmed	Work location; also used to infer remote or hybrid work in the jobs page filters.

Field	Type	Key / constraint	Description
type	String	Enum: full-time, part-time, internship, contract, remote; default full-time	Employment type.
description	String	Required	Full job description.
requiredSkills	[String]	—	Skills used as the denominator of the match score.
experienceRequired	Number	Default 0	Required experience in years.
salaryRange	String	Optional	Free-text salary range (for example "6-8 LPA").
postedBy	ObjectId	ref → User , required	Recruiter who posted the job (Posts relationship).
status	String	Enum: open, closed; default open	Only open jobs are listed publicly and scored for matches.

Table 5.3: resumes collection (model Resume)

Field	Type	Key / constraint	Description
candidate	ObjectId	ref → User , required	Owner of the resume (Uploads relationship).
fileName	String	Required	Original file name shown in the interface.
filePath	String	Required	Location of the stored file in the uploads folder.
rawText	String	—	Full text extracted from the file (or recovered by OCR).
parsedSkills	[String]	—	Skills extracted by Gemini or the rule-based parser; editable by the candidate.
parsedExperienceYears	Number	Default 0	Largest "N years" figure found in the text.
education	[{ degree, institution, year }]	Embedded, strings, default ""	Education entries.
experience	[{ title, company, duration, description }]	Embedded, strings, default ""	Work and internship entries.
certifications	[String]	—	Certificate and course names.

Table 5.4: applications collection (model Application)

Field	Type	Key / constraint	Description
job	ObjectId	ref → Job , required; part of unique index (job, candidate)	Job applied for (Receives relationship).
candidate	ObjectId	ref → User , required; part of unique index (job, candidate)	Applicant (Submits relationship). The compound index allows one application per candidate per job.
resume	ObjectId	ref → Resume, optional	Latest resume at the time of applying (Used in relationship).
matchScore	Number	Default 0 (0–100)	Skill-overlap score calculated when the application is created.

Field	Type	Key / constraint	Description
status	String	Enum: applied, under_review, shortlisted, interview, rejected, hired, withdrawn; default applied	Current stage in the hiring pipeline.

Table 5.5: interviews collection (model Interview)

Field	Type	Key / constraint	Description
application	ObjectId	ref → Application , required	Application being interviewed (Leads to relationship).
job	ObjectId	ref → Job , required	Job for the interview (Has relationship).
candidate	ObjectId	ref → User, required	Denormalised reference to the candidate.
scheduledBy	ObjectId	ref → User, required	Recruiter who scheduled the interview (denormalised).
scheduledAt	Date	Required	Date and time of the interview.
mode	String	Enum: online, in-person, phone; default online	How the interview is conducted.
notes, interviewerName, type, location, meetingLink	String	Optional (trimmed except notes)	Details shown to the candidate; type is, for example, Technical or HR.
feedback	{ technicalSkills, communication, problemSolving, comments, recommendation }	Embedded; ratings are Numbers with min 1 and max 5	Recruiter's assessment, never returned to candidates.
status	String	Enum: scheduled, completed, cancelled; default scheduled	Interview state; feedback is allowed only when completed.

Table 5.6: offerletters collection (model OfferLetter)

Field	Type	Key / constraint	Description
application	ObjectId	ref → Application , required, unique	Hired application the offer belongs to (Results in, 1:1).
candidate	ObjectId	ref → User, required	Denormalised reference to the candidate.
job	ObjectId	ref → Job, required	Denormalised reference to the job.
recruiter	ObjectId	ref → User, required	Recruiter who issued the offer.
position, company	String	Required, trimmed	Role offered and employer name.
joiningDate, offerExpiry	Date	Required	Start date and date by which the offer must be accepted.
employmentType	String	Required, trimmed	For example full-time or internship.
salary	String	Required, trimmed	Compensation offered.
workLocation, reportingTo, additionalTerms	String	Optional, trimmed	Further terms of the offer.
status	String	Enum: draft, sent; default draft	The API creates offers with status sent.
sentAt	Date	—	Time the offer was sent; updated by the resend action.

Table 5.7: conversations collection (model Conversation)

Field	Type	Key / constraint	Description
user	ObjectId	ref → User , required	Owner of the conversation (Has relationship).
userRole	String	Required, enum: candidate, recruiter, admin	Role at the time of the conversation; selects the assistant's system prompt.
messages	[{ role, content }]	Embedded; role enum: user, assistant; content required; per-message timestamps	Ordered chat history sent back to Gemini as context.

The schema reflects several deliberate design decisions. Interview feedback is embedded in the interview because it has no meaning on its own and is always read with the interview; the controller excludes it with a projection (`select("-feedback")`) when a candidate requests their interviews. Saved jobs are embedded in the user because the list is small and always belongs to one user, and the API uses an `$ne` condition in the update filter so that saving the same job twice does not create a duplicate entry. The match score is stored on the application, rather than recomputed, so that recruiters see the score that applied when the candidate applied even if the candidate later edits their skills. Finally, all status fields are enumerations validated by Mongoose, which ensures that the pipeline can only contain the stages the application understands.

CHAPTER -6 IMPLEMENTATION

This chapter describes how HireHub was implemented. Section 6.1 walks through the back-end modules and the rules each one enforces, Section 6.2 explains how the React front end renders pages and manages state, Section 6.3 documents the REST API and how the front end calls it, and Section 6.4 describes authentication and authorisation. File and function names refer to the project source code.

6.1 Module-wise Implementation

6.1.1 Authentication module

Implemented in `controllers/authController.js` and `routes/authRoutes.js`. The register handler requires name, email, password and role, checks that the role is one of candidate, recruiter or admin, and rejects an e-mail address that already exists (compared in lower case) with HTTP 409. The password is hashed with `bcrypt.hash(password, 10)` before the user is created, and the company name is stored only for recruiters. The login handler looks the user up by lower-cased e-mail and compares the password with `bcrypt.compare`; both an unknown e-mail and a wrong password return the same 401 message, "Invalid email or password", so the response does not reveal which accounts exist. Both handlers return a signed token and the user object with the password field removed by a `sanitizeUser` helper.

6.1.2 Candidate profile and resume module

Implemented in `controllers/candidateController.js` with routes under `/api/candidate`. The upload route chains `requireAuth`, `requireRole("candidate")` and Multer's `uploadResumeFile.single("resume")`. Multer stores the file in the uploads folder under a unique name made of a timestamp and a random number, accepts only `.pdf` and `.docx` extensions and limits the size to 5 MB. The `uploadResume` handler then:

1. extracts the text with `extractTextFromResume` (Section 6.1.3);
2. structures it with `parseResumeWithAI`, which uses Gemini when configured and the rule-based parser otherwise;
3. deletes the candidate's previous Resume documents and their files from disk, so that each candidate has exactly one current resume;
4. creates a new Resume document with the raw text and the extracted skills, experience years, education, experience and certifications; and
5. copies the skills and years of experience to the candidate's User document, because match scoring reads them from there.

The `updateResumeDetails` handler (`PATCH /api/candidate/resume`) lets a candidate correct the extracted fields. Each field must be an array; string lists are trimmed and de-duplicated case-insensitively, entry lists keep only the expected keys and drop completely blank rows, and an edited skills list is also written back to the User

document so that match scores update immediately. `deleteResume` removes the latest resume together with its stored file and resets the candidate's skills and experience to their empty state. `getProfile` returns the user without the password together with the latest resume, and `updateProfile` accepts a role-specific whitelist of fields (personal details for candidates; company details for recruiters) and refuses an empty name. Saved jobs are handled by `saveJob`, `unsaveJob` and `getSavedJobs`, which add to, remove from and populate the embedded `savedJobs` array.

6.1.3 Resume parsing pipeline

Resume parsing is split between `utils/resumeParser.js` and `utils/aiResumeParser.js`. Text extraction depends on the file type. For a PDF, `pdf-parse` reads the text layer; if fewer than 100 characters are found the document is treated as a scan, its first five pages are rendered to images with `pdf-to-img`, and `tesseract.js` recognises their text, keeping whichever result is longer. For a DOCX file, `mammoth.extractRawText` returns the document text. Any other type raises an error.

When `GEMINI_API_KEY` is set, `parseResumeWithAI` asks Gemini for a JSON object conforming to a response schema with `skills`, `education`, `experience` and `certifications`. The prompt instructs the model not to invent skills, to use empty strings for fields it cannot find, and to follow a worked example of the expected output. Long resumes are split into chunks and the chunk results are merged with duplicate skills and entries removed. The numeric years of experience are always computed with a regular expression that finds the largest "N years" or "N yrs" figure in the text, rather than asked of the model. If there is no key, or if Gemini fails after its retries, the same output shape is produced by the rule-based functions `parseSkillsFromText`, `parseEducationLines`, `parseExperienceEntries` and `parseCertifications`.

The rule-based parser is substantial in its own right (over 800 lines). It recognises section headings such as Education, Experience, Skills and Certifications in many spellings, matches skills against a dictionary of technologies with aliases (for example "js" and "ecmascript" map to JavaScript, "golang" to Go), detects degree names and institutions with patterns, recognises date ranges such as "Jun 2023 – Aug 2023" or "2022 – Present" to separate experience entries, and identifies certifications by keywords. Its output is heuristic, which is why candidates can review and edit it.

6.1.4 Matching module

Match scoring is a single function, `calculateMatchScore(candidateSkills, job)`, in `utils/matchScore.js`. It lower-cases both lists, counts how many of the job's required skills appear in the candidate's skill set, and returns the percentage rounded to a whole number between 0 and 100. A job that lists no required skills receives a neutral score of 50. The function is used in two places: `GET /api/matches` scores every open job for the logged-in candidate and returns them sorted from the highest score to the lowest, and `applyToJob` stores the score on the new application.

Listing 6.1: utils/matchScore.js

```
// Returns a 0-100 score based on overlap between candidate skills and job's required skills
export function calculateMatchScore(candidateSkills = [], job) {
  const requiredSkills = (job.requiredSkills || []).map((s) => s.toLowerCase());
  const candidateSet = new Set(candidateSkills.map((s) => s.toLowerCase()));

  if (requiredSkills.length === 0) return 50; // neutral score if job lists no required skills

  const matched = requiredSkills.filter((skill) => candidateSet.has(skill));
  const skillScore = (matched.length / requiredSkills.length) * 100;

  return Math.round(Math.min(100, Math.max(0, skillScore)));
}
```

For example, a job requiring JavaScript, React, Node.js and MongoDB scores 75% for a candidate whose resume contains JavaScript, React and MongoDB, which is exactly the result observed in test case TC-21 in Chapter 8.

6.1.5 Jobs module

Implemented in `controllers/jobController.js`. `GET /api/jobs` is public and returns open jobs, newest first, with the recruiter's name and company populated; it supports optional query parameters `q` (title), `location`, `type` and `experience=fresher`. `POST /api/jobs` allows only recruiters and administrators, requires a title, company and description, and records the logged-in recruiter as `postedBy`. `PATCH /api/jobs/:id` updates only whitelisted fields, refuses edits by recruiters who did not post the job, re-checks the required fields, and returns HTTP 400 when Mongoose rejects an invalid enumeration value (for example an unknown job type). Closing and reopening a job is an update of its `status` field.

6.1.6 Applications module

Implemented in `controllers/applicationController.js`. `applyToJob` allows only candidates, returns 404 for an unknown job and 409 if the candidate has already applied, and creates the application with the candidate's latest resume and the calculated match score. `getApplications` returns different data for each role: a candidate receives their own applications, a recruiter receives the applications for jobs they posted, and an administrator receives all applications; in every case the job, candidate and resume are populated and the newest come first. `updateApplicationStatus` accepts only the stages a recruiter may set (`applied`, `under_review`, `shortlisted`, `interview`, `rejected`, `hired`) and checks job ownership. `withdrawApplication` lets candidates delete only their own applications.

6.1.7 Interviews module

Implemented in `controllers/interviewController.js`. `scheduleInterview` requires an `applicationId` and `scheduledAt`, checks that the recruiter owns the job, copies the job and candidate references from the application into the interview, and moves the application's status to `interview`. `getMyInterviews` returns a candidate's interviews with the `feedback` field excluded, and a recruiter's or administrator's interviews with the job, candidate and application status populated, sorted by date.

`updateInterview` reschedules (rejecting invalid dates) and changes the status to scheduled, completed or cancelled. `submitFeedback` is accepted only after the interview has been marked completed and requires three whole-number ratings from 1 to 5 for technical skills, communication and problem solving, plus optional comments and a recommendation.

6.1.8 Offer letters module

Implemented in `controllers/offerController.js`, `createOffer` requires an application whose status is hired, rejects a second offer for the same application with 409, copies the whitelisted offer fields from the request, and stores the offer with status `sent` and the current time as `sentAt`. `getOfferForApplication` returns the offer (or `null` if none exists yet) to the candidate who applied or to the recruiter who owns the job. `getCandidateOffers` lists a candidate's sent offers, newest first, which drives the congratulations banner on the candidate's home page. `resendOffer` updates the sent time; it does not send an e-mail.

6.1.9 Recruiter, administrator and chatbot modules

`controllers/recruiterController.js` provides two read-only endpoints for recruiters. `getRecruiterStats` loads the recruiter's jobs and their applications and returns, per job, the number of applicants and the counts in the under-review, shortlisted, interview and hired stages, together with overall totals including the number of active (open) jobs. `getApplicants` groups applications by candidate, so that a candidate who applied to several of the recruiter's jobs appears once with all their applications listed, and attaches each candidate's most recent parsed resume. `controllers/adminController.js` provides `getAnalytics`, which runs five `countDocuments` queries in parallel and a `$group` aggregation of applications by status. `controllers/chatbotController.js` stores each exchange in a `Conversation` document, continues an existing conversation when a valid `conversationId` belonging to the user is supplied, and calls Gemini with the role-specific system prompt, or returns the fallback reply when no key is configured.

6.2 Front-end Logic (UI Rendering, State Management)

Rendering and page structure

The React application is composed of function components. App renders the `Navbar`, a `Routes` table and a footer. Pages for logged-in users are wrapped in `DashboardLayout`, which draws the sidebar for the user's role (Jobs, Applications, Interviews and Profile, with role-specific paths) and a Logout button, and renders the page content beside it. Larger pages are composed of smaller components; for example, the candidate's Applications page uses `StatusChip` and `ApplicationDetail`, the recruiter's Applications page uses `ApplicationStageTrack`, `ResumeSummary` and `HiringActions`, and both Interviews pages use a shared `InterviewCalendar`. Offer letters are displayed by `OfferLetterView` inside a modal dialog and can be downloaded as a text file generated in the browser with a `Blob`.

The home route renders a different page depending on who is logged in. `Home` reads the stored authentication record and renders `CandidateHome` for candidates, `RecruiterHome` for recruiters and the public landing page

for everyone else. `CandidateHome` shows a greeting with an Upload/Update Resume button, offer-letter banners, statistics tiles, a Profile Boost card summarising the parsed resume, the editable Resume Insights section, the top four AI matches, a profile-completion card and an application-progress timeline. `RecruiterHome` shows a Post a New Job button, KPI tiles (active jobs, total applicants, interviews and hires), recent applications, upcoming interviews and the recruiter's jobs.

State management

Each page keeps its own data in component state created with `useState`, loads it in a `useEffect` hook when the page mounts, and shows loading and error messages while doing so. Data that is needed together is loaded in parallel with `Promise.all` (for example the candidate home page loads the profile and applications together, and then loads matches only if a resume exists). Effects use a `cancelled` flag in their clean-up function so that a response arriving after the user has navigated away does not update an unmounted component. After a successful change, such as a resume upload or a status update, the page either replaces the affected item in state with the server's response or reloads the relevant data, so the screen always reflects what the server stored.

Derived views are computed during rendering rather than stored. On the candidate jobs page, the list of all open jobs and the list of matches are fetched once; the search text, filters (job type, work mode, experience band, minimum salary, location and skill), the selected tab (All, Recommended or Saved) and the sort order (newest, oldest, title or best match) are kept in state, and the visible list is recalculated from them on every render. Work mode is inferred from the job type and location text (remote, hybrid or on-site), and the Recommended tab shows matches with a score above zero. Profile completion is computed as the percentage of eight items that are present: resume, skills, title, phone, location, LinkedIn, GitHub and an About section.

Authentication state is the only global state. `saveAuth` stores the token, role, name and e-mail in `localStorage` under `hirehub_auth` after login or registration, `getAuth` reads it, and `clearAuth` removes it on logout. Because the record lives in storage rather than in a React context, every component reads the current user directly, and a page reload keeps the user logged in until the token expires.

6.3 API Integration

All communication with the back end goes through `src/api.js`. A shared helper, `parseJsonOrThrow`, reads the JSON body of every response and throws an `Error` carrying the server's message when the status is not successful, so that each page can display the server's own explanation (for example "You have already applied to this job"). Each exported function corresponds to one endpoint, sets the method and headers, and includes the bearer token for protected routes. Listing 6.2 shows the pattern.

Listing 6.2: Request pattern in src/api.js (abridged)

```

const API_BASE_URL = import.meta.env.VITE_API_URL || "http://localhost:8000/api";

async function parseJsonOrThrow(response, fallbackMessage) {
  let data = {};
  try { data = await response.json(); } catch { /* no JSON body */ }
  if (!response.ok) throw new Error(data.message || fallbackMessage);
  return data;
}

export async function uploadResume(file, token) {
  const form = new FormData();
  form.append("resume", file);
  const response = await fetch(`${API_BASE_URL}/candidate/resume`, {
    method: "POST",
    headers: { Authorization: `Bearer ${token}` },
    body: form,
  });
  return parseJsonOrThrow(response, "Resume upload failed");
}

```

Table 6.1 lists every route registered by the back end, with the roles allowed by the route's middleware. Routes marked "any" require a valid token but no particular role; ownership rules applied inside the controllers are described in Section 6.4.

Table 6.1: REST API endpoints (31 route handlers and a health check)

Method	Endpoint	Access	Purpose
GET	/api/health	Public	Health check; returns {status: "ok"}.
POST	/api/auth/register	Public	Create an account and return a token.
POST	/api/auth/login	Public	Log in and return a token.
GET	/api/jobs	Public	List open jobs (optional q, location, type, experience).
POST	/api/jobs	Any (recruiter/admin in controller)	Create a job.
PATCH	/api/jobs/:id	Recruiter, admin	Edit, close or reopen a job.
POST	/api/candidate/resume	Candidate	Upload and parse a resume (multipart field <code>resume</code>).
PATCH	/api/candidate/resume	Candidate	Correct the parsed resume fields.
DELETE	/api/candidate/resume	Candidate	Remove the latest resume.
GET	/api/candidate/profile	Candidate, recruiter	Profile and latest resume.
PUT / PATCH	/api/candidate/profile	Candidate, recruiter	Update profile fields (two routes, same handler).
GET	/api/candidate/saved-jobs	Candidate	List saved jobs.
POST	/api/candidate/saved-jobs/:job_id	Candidate	Save a job.
DELETE	/api/candidate/saved-jobs/:job_id	Candidate	Remove a saved job.
GET	/api/matches	Candidate	All open jobs with match scores, best first.
POST	/api/jobs/:job_id/apply	Any (candidate in controller)	Apply to a job.
GET	/api/applications	Any (filtered by role)	List applications visible to the user.
PATCH	/api/applications/:id/status	Recruiter, admin	Change an application's stage.
DELETE	/api/applications/:id	Candidate	Withdraw an application.
POST	/api/interviews	Recruiter, admin	Schedule an interview.
GET	/api/interviews	Candidate, recruiter, admin	List interviews visible to the user.

Method	Endpoint	Access	Purpose
PATCH	/api/interviews/:id	Recruiter, admin	Reschedule or change interview status.
POST	/api/interviews/:id/feedback	Recruiter, admin	Record interview feedback.
POST	/api/offers	Recruiter, admin	Create an offer letter for a hired application.
GET	/api/offers/candidate	Candidate	List the candidate's offers.
GET	/api/offers/application/:applicationId	Any (ownership in controller)	Offer for one application.
PATCH	/api/offers/:id/resent	Recruiter, admin	Mark an offer as re-sent.
GET	/api/recruiter/stats	Recruiter, admin	Per-job and total hiring statistics.
GET	/api/recruiter/applicants	Recruiter, admin	Applicants grouped by candidate with latest resume.
GET	/api/admin/analytics	Admin	Platform totals and applications by status.
POST	/api/chatbot/message	Any	Send a message to the AI assistant.

The front end uses all of these endpoints except three: the administrator analytics endpoint and the chatbot endpoint, which have no user-interface component yet, and the `PATCH` variant of the profile route (the client uses `PUT`). The jobs pages call `GET /api/jobs` without query parameters and apply search and filters in the browser, as described in Section 6.2. Resume files are opened by building a URL to the server's `/uploads` folder from the stored file path with the helper `resumeFileUrl`.

6.4 Authentication and Authorisation

Authentication

HireHub uses stateless authentication with JSON Web Tokens. After a successful registration or login, the server signs a token whose payload contains the user's id, role and e-mail, using the secret in `JWT_SECRET` and an expiry taken from `JWT_EXPIRES_IN` (seven days by default). The client stores the token and sends it on every protected request in the header `Authorization: Bearer <token>`. Passwords are never stored or returned in plain text; they are hashed with `bcrypt` using a cost factor of 10, and the hash is removed from every user object returned by the API.

Listing 6.3: middleware/auth.js

```
export function requireAuth(req, res, next) {
  const header = req.headers.authorization || "";
  const token = header.startsWith("Bearer ") ? header.slice(7) : null;
  if (!token) {
    return res.status(401).json({ message: "Missing or malformed Authorization header" });
  }
  try {
    const payload = jwt.verify(token, process.env.JWT_SECRET);
    req.user = payload; // { id, role, email }
    next();
  } catch (err) {
    return res.status(401).json({ message: "Invalid or expired token" });
  }
}

// Usage: requireRole("recruiter", "admin")
export function requireRole(...allowedRoles) {
  return (req, res, next) => {
    if (!req.user || !allowedRoles.includes(req.user.role)) {
      return res.status(403).json({ message: "You do not have access to this resource" });
    }
    next();
  };
}
```

Authorisation

Authorisation is applied in two layers. The first is role-based: `requireRole` is attached to routes and rejects users whose role is not in the allowed list with HTTP 403 (Table 6.1). The second is ownership-based and is implemented in the controllers, because it depends on the data being accessed rather than on the route. The ownership rules are:

- A recruiter can edit only jobs whose `postedBy` is their own id.
- A recruiter sees, and can change the status of, only applications for jobs they posted; a candidate sees only their own applications and can withdraw only those.
- A recruiter can schedule interviews, change them and record feedback only for jobs they posted; a candidate sees only their own interviews, without the feedback.
- Offer letters can be created and resent only by the recruiter who owns the job; an offer can be viewed only by that recruiter or by the candidate who applied.
- A chatbot conversation is continued only if it belongs to the requesting user.
- Administrators bypass ownership checks and can see all applications, applicants and interviews.

On the client side, the `ProtectedRoute` component redirects visitors without a stored token to `/login` and redirects users whose role is not allowed on a route to their own dashboard. This improves the user experience but is not relied on for security: because every rule above is enforced by the API, a user who altered the client could still not read or change another user's data. `ProtectedRoute`, the navigation bar and the role helpers compare the stored role case-insensitively, since the API returns roles in lower case.

CHAPTER -7 FEATURES

7.1 List of Main Features

Table 7.1 lists the main features of HireHub, the role that uses each one and its status in the current implementation. All features marked "In user interface" are reachable from the React application; the two features marked "API only" are implemented and tested on the back end but do not yet have a screen.

Table 7.1: Main features of HireHub

No.	Feature	Used by	Status
F1	Registration, login and role-aware home page	All	In user interface
F2	Resume upload with automatic parsing (PDF/DOCX, OCR, Gemini or rule-based)	Candidate	In user interface
F3	Resume Insights review and correction	Candidate	In user interface
F4	Job search, filters and sorting	Candidate	In user interface
F5	AI job matches and recommendations	Candidate	In user interface
F6	Saved jobs	Candidate	In user interface
F7	Applying, application tracking and withdrawal	Candidate	In user interface
F8	Candidate interview calendar	Candidate	In user interface
F9	Offer letter notification, viewing and download	Candidate	In user interface
F10	Candidate and recruiter profiles	Candidate, recruiter	In user interface
F11	Job posting and management	Recruiter	In user interface
F12	Applicant review with match scores and parsed resumes	Recruiter	In user interface
F13	Application status pipeline	Recruiter	In user interface
F14	Interview scheduling, updates and feedback	Recruiter	In user interface
F15	Offer letter creation and resend	Recruiter	In user interface
F16	Hiring statistics dashboard	Recruiter	In user interface
F17	Platform analytics	Administrator	API only (UI page shows static figures)
F18	AI career and recruiting assistant	All	API only
F19	Responsive layout with mobile navigation	All	In user interface

7.2 How Each Feature Works (User Perspective + Technical Description)

F1 – Registration, login and role-aware home page

User perspective. A visitor opens the landing page, chooses Get Started, enters a name, e-mail address and password, and selects whether they are a candidate or a recruiter (the form also offers an Admin option; see Section 10.3). Recruiters add their company details afterwards on their Profile page. The login form asks for a name or username as well as the e-mail address and password, although only the e-mail address and password are checked. After registering or logging in, the user is taken to the home page, which now shows their own overview instead of the marketing page, and the navigation bar shows Dashboard and Logout buttons.

Technical description. The form calls `registerUser` or `login` in `api.js`. The API validates the fields, hashes the password, signs a JWT and returns it with the user object. `saveAuth` stores the token, role, name and e-mail in `localStorage`. The `Home` component reads this record on every navigation and renders `CandidateHome`, `RecruiterHome` or the landing page accordingly; logging out clears the record and returns to `/`.

F2 – Resume upload with automatic parsing

User perspective. On the home page or the Profile page, a candidate clicks Upload Resume and chooses a PDF or Word (DOCX) file. After a short wait the Profile Boost card reports that the resume is AI-ready, shows the file name, and the Resume Insights section lists the skills, experience, education and certifications that were found. Uploading again replaces the previous resume.

Technical description. The file is sent as multipart form data to `POST /api/candidate/resume`. Multer validates the extension and size and stores the file; the controller extracts text (with OCR for scanned PDFs), parses it with Gemini or the rule-based parser, replaces any earlier resume and its file, saves the new Resume document and copies the skills and experience years to the user. The response contains the structured fields, and the page reloads the profile and the matches so that recommendations reflect the new skills immediately.

F3 – Resume Insights review and correction

User perspective. Because automatic parsing can miss or misread information, the Resume Insights section has an Edit button. The candidate can add or remove skills, edit education and experience rows and certifications, and save the changes.

Technical description. `ResumeInsightsEditor` sends the edited arrays to `PATCH /api/candidate/resume`. The server validates that each field is an array, trims and de-duplicates the values, drops blank rows, saves the resume and, when skills change, updates `User.skills`. The page then refreshes the profile and matches, so an added skill raises the corresponding match scores at once.

F4 – Job search, filters and sorting

User perspective. The Jobs page lists open jobs as cards showing the title, company, location, employment type and work mode, salary, required skills and, for candidates with a resume, the match percentage. The candidate can search by keyword and location, filter by job type, work mode (remote, hybrid or on-site), experience band, minimum salary, location and skill, and sort by newest, oldest, title or best match. Selecting a job opens a details page with the full description.

Technical description. The page fetches all open jobs from the public `GET /api/jobs` endpoint once and applies search, filters and sorting in the browser, recomputing the visible list on every render from the values held in state. Work mode is inferred from the job's type and location text, and the salary filter reads the upper bound of the salary range text. Search terms entered on the landing page are passed to the Jobs page as a `q` query parameter in the URL.

F5 – AI job matches and recommendations

User perspective. After uploading a resume, the candidate sees a Best Match tile and an AI Matches list of the four highest-scoring jobs on the home page, a Recommended tab on the Jobs page, and a match percentage on each job card and job details page. Jobs that require more of the candidate's skills appear first.

Technical description. GET /api/matches applies calculateMatchScore to every open job using the skills stored on the candidate's user document and returns the jobs sorted by score. The front end keeps the scores in a map keyed by job id to annotate cards and to implement the best-match sort, and the Recommended tab shows matches with a score above zero.

F6 – Saved jobs

User perspective. A candidate can bookmark a job with the save icon on its card and later find it under the Saved tab of the Jobs page; clicking the icon again removes it.

Technical description. POST and DELETE /api/candidate/saved-jobs/:job_id add to and remove from the embedded savedJobs array on the user document. The add operation uses an update filter that matches only when the job is not already saved, so repeated clicks cannot create duplicates; GET /api/candidate/saved-jobs populates the jobs, drops entries whose job no longer exists and sorts by the time saved. The Saved Jobs tile on the candidate home page shows the current number of saved jobs, loaded with the same endpoint.

F7 – Applying, application tracking and withdrawal

User perspective. The candidate clicks Apply on a job. The Applications page lists every application with a status chip (Applied, Under Review, Shortlisted, Interview, Selected or Not selected); selecting one shows its details, the match score, a stage tracker and, if the candidate is hired, the offer letter. An application can be withdrawn while it is still in progress. The home page summarises progress in a Your Journey timeline.

Technical description. Applying calls POST /api/jobs/:job_id/apply, which stores the current match score and latest resume and is protected against duplicates by both a controller check and the unique compound index on job and candidate. The list comes from GET /api/applications, filtered to the candidate by the server. Withdrawal calls DELETE /api/applications/:id, which verifies that the application belongs to the candidate before deleting it.

F8 – Candidate interview calendar

User perspective. The Interviews page shows a monthly calendar with the candidate's interviews and a list of upcoming and past interviews. Each interview shows the job, date and time, mode (online, in-person or phone), interview type, interviewer, meeting link or location, notes from the recruiter and general preparation tips.

Technical description. The page calls GET /api/interviews, for which the server returns only the candidate's own interviews and excludes the recruiter's feedback with a projection. InterviewCalendar groups

interviews by day for the visible month, and `isUpcomingInterview` separates upcoming scheduled interviews from completed, cancelled or past ones.

F9 – Offer letter notification, viewing and download

User perspective. When a recruiter sends an offer, the candidate's home page shows a congratulations banner naming the position and company with a View Offer Letter button. The letter states the position and company, lists the work location, employment type, joining date, salary or stipend and reporting manager, includes the date by which the offer must be accepted and any additional terms, and can be downloaded as a text file.

Technical description. The home page calls `GET /api/offers/candidate` and renders a banner for each sent offer; the Applications page calls `GET /api/offers/application/:applicationId` for a hired application. `offerLetterText` formats the letter and `downloadOfferLetter` creates a Blob in the browser and triggers a download, so no file is generated on the server.

F10 – Candidate and recruiter profiles

User perspective. Candidates maintain a professional title, phone number, location, LinkedIn and GitHub links and an About section, and manage their resume; a profile-completion percentage encourages them to fill in missing items. Recruiters maintain their name, phone, location and company details (company name, industry, website and description), and see a hiring overview of their own activity.

Technical description. Both profiles use `GET /api/candidate/profile` and `PUT /api/candidate/profile`. The server accepts only the fields allowed for the user's role, trims them and rejects an empty name. Profile completion is calculated on the client from eight items.

F11 – Job posting and management

User perspective. A recruiter clicks Post a New Job on the home page or the Jobs page and fills in the title, company, location, employment type, description, required skills (comma-separated), experience required and salary range. The recruiter's Jobs page lists every job with its applicant counts by stage, and each job can be edited, closed or reopened.

Technical description. `PostJobModal` validates the required fields in the browser and calls `POST /api/jobs` or, when editing, `PATCH /api/jobs/:id`. Closing or reopening a job sends only the new status. The per-job counts come from `GET /api/recruiter/stats`. Closed jobs disappear from the public list and from match scoring because both only consider open jobs.

F12 – Applicant review with match scores and parsed resumes

User perspective. The recruiter's Applications page lists applicants for the recruiter's jobs and can be filtered by job, stage and experience and searched by name. Selecting an applicant shows their e-mail address, the match

score for the selected job, the stage tracker, the candidate's skills and education from the parsed resume, and a link to open the original resume file.

Technical description. The page loads `GET /api/applications` and `GET /api/recruiter/applicants` in parallel. The `applicants` endpoint groups applications by candidate and attaches the latest resume; `ResumeSummary` renders it, falling back to the skills on the user record. The resume file link is built by `resumeFileUrl` from the stored file path and opens the file served statically under `/uploads`.

F13 – Application status pipeline

User perspective. For each application the recruiter moves the candidate through the stages Applied, Under Review, Shortlisted, Interview and finally Hired or Rejected. The candidate sees the change on their Applications page and home-page timeline.

Technical description. Status buttons call `PATCH /api/applications/:id/status`. The server accepts only the six recruiter-settable stages and checks that the recruiter owns the job; the updated application is returned with its job, candidate and resume populated so the page can replace it in state. `ApplicationStageTrack` maps each status to its position in the pipeline for display.

F14 – Interview scheduling, updates and feedback

User perspective. From an applicant, the recruiter chooses Schedule Interview, which opens the Interviews page with that application selected. The recruiter sets the date and time, interview type (Screening, Technical, HR, Managerial or Final), mode, meeting link or location, interviewer and notes. Interviews appear on a calendar and can be rescheduled, cancelled or marked completed. After completion, the recruiter records ratings from 1 to 5 for technical skills, communication and problem solving, comments and a recommendation (Strong Hire, Hire, Maybe or No Hire).

Technical description. The schedule form posts to `POST /api/interviews`, which also sets the application's status to `interview`. Only applications that are not rejected, hired or withdrawn are offered for scheduling. Updates use `PATCH /api/interviews/:id` and feedback uses `POST /api/interviews/:id/feedback`; the server refuses feedback for an interview that is not completed and validates the ratings.

F15 – Offer letter creation and resend

User perspective. When an application is marked Hired, a Hiring Actions panel appears with a Create Offer Letter button. The recruiter completes the offer form, reviews a preview of the letter and then sends it; afterwards the panel shows the letter with options to view, download or resend it.

Technical description. `HiringActions` loads any existing offer with `GET /api/offers/application/:applicationId`, creates one with `POST /api/offers` and resends with `PATCH /api/offers/:id/resend`. The server allows offers only for hired applications and only one per application, enforced by both a check and a unique index.

F16 – Hiring statistics dashboard

User perspective. The recruiter's home page shows the number of active jobs, total applicants, interviews and hires, together with recent applications, upcoming interviews and a list of the recruiter's jobs with applicant counts.

Technical description. `RecruiterHome` loads `GET /api/recruiter/stats`, `GET /api/applications` and `GET /api/interviews` in parallel. The statistics are calculated on the server from the recruiter's jobs and their applications.

F17 – Platform analytics (API only)

User perspective. An administrator would see counts of candidates, recruiters, jobs, applications and interviews and a breakdown of applications by status. The current Admin page in the user interface displays fixed sample figures and is not yet connected to this data.

Technical description. `GET /api/admin/analytics` is restricted to the `admin` role and combines five `countDocuments` queries executed in parallel with an aggregation pipeline that groups applications by status (verified in test case TC-43).

F18 – AI career and recruiting assistant (API only)

User perspective. The assistant is intended to answer questions about job searching, resumes and interview preparation for candidates, and about job postings, screening and interviews for recruiters. A chat interface has not yet been added to the front end.

Technical description. `POST /api/chatbot/message` accepts a message and an optional conversation id, stores both the user's message and the reply in a `Conversation` document, and sends the conversation history to Gemini with a role-specific system instruction. Without an API key it returns a clearly labelled fallback reply (verified in test case TC-45).

F19 – Responsive layout with mobile navigation

User perspective. The application can be used on phones and tablets: the navigation collapses into a menu button, and dashboard grids and cards stack into a single column on narrow screens.

Technical description. Layout is implemented with CSS grid and flexbox in `styles.css`, with media queries that change column counts and hide the dashboard sidebar on small screens; the `Navbar` component toggles the mobile menu with a state variable and closes it on navigation.

CHAPTER -8 TESTING

Testing aimed to confirm that every REST endpoint behaves as specified for valid input, rejects invalid input with the correct status code, and enforces the role and ownership rules described in Chapter 6; that the modules work together across a complete hiring flow; and that the main user journeys work in a real browser. This chapter explains how the API and the user interface were tested, which tools were used, and the test cases with their actual results. All results reported in this chapter come from test runs carried out on 2 October 2026 against the final version of the code, including the corrections described in Section 10.2.

8.1 Postman Testing (Frontend/Backend)

The project repository does not contain a Postman collection (an earlier version of the back-end README referred to one, and the README has since been corrected), so no Postman results are claimed in this report. Instead, the API was exercised with a scripted HTTP client that sends exactly the requests a Postman collection would contain and records each response automatically (Section 8.3). This made the run repeatable: the full suite can be executed again in a few seconds against a fresh database and produces the same results.

Every API test case in Table 8.3 can be reproduced manually in Postman (or any HTTP client) using the information in the table and the conventions below, which also serve as the specification for building a Postman collection for the project:

- **Base URL:** `http://localhost:8000/api` for the development server (the isolated test runs used port 8100).
- **Content type:** JSON bodies are sent with the header `Content-Type: application/json`. Resume uploads are sent as `multipart/form-data` with the file in a field named `resume`.
- **Authentication:** call `POST /auth/register` or `POST /auth/login`, copy the token from the response and send it on protected requests in the header `Authorization: Bearer <token>`. In Postman this is conveniently stored as a collection variable.
- **Accounts:** the role-based tests need four accounts: a candidate, a recruiter, a second recruiter (to test ownership) and an administrator.
- **Identifiers:** ids returned by earlier requests (job, application, interview and offer ids) are used in the paths of later requests, so the cases are executed in the order listed.

The front end was tested separately through the browser (Section 8.2 and Table 8.4), because Postman exercises only the API. The API client in `src/api.js` sends the same requests as the API tests, so the API tests also validate the contract on which the React application relies.

8.2 Integration Testing

Integration testing was carried out at two levels. At the API level, the test cases were written as one ordered scenario in which each step depends on the data created by the previous ones, rather than as isolated unit tests:

1. register a candidate, two recruiters and an administrator, and log in;
2. create two jobs as the first recruiter (Frontend Developer requiring JavaScript, React, Node.js and MongoDB; Backend Developer requiring Node.js, Express and MongoDB);
3. upload a resume as the candidate and verify the parsed fields and the match scores for both jobs;
4. correct the parsed skills and apply to the Frontend job;
5. review the application as its recruiter, change its status, schedule an interview, complete it and record feedback;
6. mark the application hired, create an offer letter and retrieve it as the candidate;
7. check recruiter statistics, administrator analytics, saved jobs, the chatbot, withdrawal of a second application, removal of the resume (including its file) and the upload size limit.

Integration points verified by this scenario include: the resume upload updating both the Resume and User collections and thereby changing match scores (TC-20, TC-21, TC-23, TC-24); scheduling an interview moving the related application to the Interview stage (TC-32); offer creation depending on the application's status and on the unique index on the application (TC-38 to TC-40); recruiter statistics aggregating jobs and applications (TC-42); access rules that depend on data owned by other users (TC-16, TC-28); and resume removal deleting both the database record and the stored file (TC-47, TC-50).

At the user-interface level, the production build of the React application was opened in Google Chrome and driven by an automated browser script (Playwright) that performs the same actions as a user: filling in forms, clicking buttons, choosing filter values, uploading a file and downloading the offer letter. Two browser sessions were used at once, one logged in as a candidate and one as a recruiter, so that an action taken by one role (for example shortlisting an application) could be checked from the other role's screen. These cases are listed in Table 8.4.

Isolation of test data. No test used the project's real database. A temporary MongoDB instance was started in memory with `mongodb-memory-server`, and a separate copy of the back-end code was started against it on port 8100 with test-only environment values and no Gemini API key, so that resume parsing used the rule-based parser and the chatbot returned its fallback reply. For the user-interface tests the front end was built into a temporary folder with `VITE_API_URL` pointing to this test API and served on port 5180, and the database was filled with the project's own demonstration jobs (`scripts/seedJobs.js`, about 150 jobs). All test accounts used reserved `example.test` e-mail addresses. The temporary database was discarded when the test server stopped, and uploaded test files were written to a temporary folder outside the project.

Front-end build. The React application was compiled with Vite's production build (writing its output outside the project folder). The build completed successfully, transforming 1,993 modules, and produced a JavaScript bundle

of about 506 kB (151 kB compressed), for which Vite printed its standard advisory that chunks larger than 500 kB could be split. No build errors were reported.

8.3 Tools Used for Testing

Table 8.1: Testing tools

Tool	Purpose in testing
Node.js (v24) with the built-in <code>fetch</code> , <code>FormData</code> and <code>Blob</code> APIs	API test script that sends each HTTP request, including multipart resume uploads, and compares the status code and response body with the expected result.
Playwright (playwright-core) with Google Chrome	Automated browser tests of the user interface (Table 8.4), including file upload, dialogs and file download.
mongodb-memory-server	Temporary, isolated MongoDB server so that tests never touch the real database.
Mongoose (from the back end's dependencies)	Dropping the temporary database before a run so that every run starts from an empty state.
Project seed script (<code>npm run seed:jobs</code>)	Loading demonstration jobs into the temporary database for the user-interface tests.
Microsoft Word	Producing realistic test resumes in PDF and DOCX format.
PyMuPDF	Producing a PDF with a different internal structure to test the robustness of PDF parsing (TC-49).
Vite build	Compiling the front end to detect build-time errors and to produce the build used by the browser tests.
Server console log	Confirming the cause of failures, such as the PDF parsing error in TC-49.

The API test data consisted of four accounts, two jobs, a one-page resume describing a candidate with JavaScript, React, MongoDB, Git, HTML and CSS skills, a B.Tech degree, one internship and one certification (saved as both PDF and DOCX), a plain text file used as an invalid upload, a 6 MB file used to test the size limit, and a PDF of the same resume generated by a different library.

8.4 Test Cases and Results

Table 8.2 summarises the API results by module, and Table 8.3 lists every executed API test case with its request, expected result, actual result and status. Of 51 executed API cases, 50 passed and 1 failed. Table 8.4 lists the user-interface test cases; all 17 were executed in Chrome and 17 passed.

Table 8.2: Summary of test results

Module	Executed	Passed	Failed
System	2	2	0
Auth	9	9	0
Jobs	6	6	0
Resume	10	9	1
Matching	1	1	0
Applications	8	8	0
Interviews	7	7	0
Offers	4	4	0

Module	Executed	Passed	Failed
Recruiter	1	1	0
Admin	1	1	0
Saved jobs	1	1	0
Chatbot	1	1	0
User interface (Table 8.4)	17	17	0
Total	68	67	1

Table 8.3: API test cases and results (executed 2 October 2026)

ID	Endpoint	Request / input	Expected result	Actual result	Status
TC-01	GET /api/health	No body	200 with {status: "ok"}	200 {status: "ok"}	Pass
TC-02	GET /api/unknown-route	No body	404 "Route not found"	404 "Route not found"	Pass
TC-03	POST /api/auth/register	Body without password and role	400 validation message	400 "name, email, password and role are required"	Pass
TC-04	POST /api/auth/register	role = "superuser"	400 invalid role	400 "role must be candidate, recruiter or admin"	Pass
TC-05	POST /api/auth/register	Valid candidate details	201 with JWT token; password not returned	201, token issued: true, password field in response: false	Pass
TC-06	POST /api/auth/register	Same email again	409 duplicate account	409 "An account with this email already exists"	Pass
TC-07	POST /api/auth/register	Valid recruiter details with company	201 with JWT token	201, role "recruiter", company "Example Corp"	Pass
TC-08	POST /api/auth/login	Correct email, wrong password	401 "Invalid email or password"	401 "Invalid email or password"	Pass
TC-09	POST /api/auth/login	Correct credentials (email in upper case)	200 with JWT token	200, token issued: true	Pass
TC-10	GET /api/applications	No Authorization header	401 missing header	401 "Missing or malformed Authorization header"	Pass
TC-11	GET /api/applications	Authorization: Bearer invalid.token.value	401 invalid token	401 "Invalid or expired token"	Pass
TC-12	POST /api/jobs	Candidate token, valid job body	403 only recruiters can post	403 "Only recruiters can post jobs"	Pass
TC-13	POST /api/jobs	Recruiter token, description missing	400 required fields	400 "title, company and description are required"	Pass
TC-14	POST /api/jobs	Recruiter token, valid job (4 required skills)	201 job created, status open	201, status "open"	Pass
TC-15	GET /api/jobs?q=frontend	Public request, no token	200 list containing the new job	200, 1 job(s) returned, new job included: true	Pass
TC-16	PATCH /api/jobs/:id	Different recruiter edits the job	403 not the owner	403 "You can only edit jobs you posted"	Pass
TC-17	PATCH /api/jobs/:id	Owner sets type = "freelance"	400 enum validation error	400 (validation error)	Pass
TC-18	POST /api/candidate/resume	Multipart request with no file	400 no file uploaded	400 "No resume file uploaded (field name must be 'resume')"	Pass
TC-19	POST /api/candidate/resume	Upload a .txt file	400 "Only .pdf and .docx resumes are accepted"	400 "Only .pdf and .docx resumes are accepted"	Pass
TC-20	POST /api/candidate/resume	Upload a text-based PDF resume exported from MS Word (no Gemini key: rule-based parser)	201 with extracted skills, education, experience, certifications	201; skills [JavaScript, React, MongoDB, Git, HTML, CSS, Web Development, REST APIs]; education 1; experience 1; certifications 1; experience years 2	Pass

ID	Endpoint	Request / input	Expected result	Actual result	Status
TC-21	GET /api/matches	Candidate token after resume upload	200; jobs ranked by % of required skills matched	200; Frontend Developer (Test): 75%; Backend Developer (Test): 33%	Pass
TC-22	PATCH /api/candidate/resume	parsedSkills sent as a string	400 must be an array	400 "parsedSkills must be an array"	Pass
TC-23	PATCH /api/candidate/resume	parsedSkills with duplicates and blanks, adding Node.js	200; list de-duplicated, profile skills updated	200; skills [JavaScript, React, MongoDB, Git, HTML, CSS, Web Development, REST APIs, Node.js]	Pass
TC-24	POST /api/jobs/:id/apply	Candidate applies to the Frontend job	201 with stored matchScore	201; matchScore 100; status "applied"	Pass
TC-25	POST /api/jobs/:id/apply	Same candidate applies again	409 already applied	409 "You have already applied to this job"	Pass
TC-26	POST /api/jobs/:id/apply	Recruiter token	403 only candidates can apply	403 "Only candidates can apply to jobs"	Pass
TC-27	GET /api/applications	Recruiter token	200; only applications for this recruiter's jobs	200; 1 application(s)	Pass
TC-28	GET /api/applications	Other recruiter's token	200; empty list (no access to other jobs)	200; 0 application(s)	Pass
TC-29	PATCH /api/applications/:id/status	status = "withdrawn" (not recruiter-settable)	400 invalid status	400 (status list returned)	Pass
TC-30	PATCH /api/applications/:id/status	status = "shortlisted"	200; status updated	200; status "shortlisted"	Pass
TC-31	POST /api/interviews	applicationId without scheduledAt	400 required fields	400 "applicationId and scheduledAt are required"	Pass
TC-32	POST /api/interviews	Valid online interview with meeting link	201; application moves to "interview"	201; application status now "interview"	Pass
TC-33	POST /api/interviews/:id/feedback	Feedback while still "scheduled"	400 must be completed first	400 "Mark the interview as completed before adding feedback"	Pass
TC-34	PATCH /api/interviews/:id	status = "completed"	200; interview completed	200; status "completed"	Pass
TC-35	POST /api/interviews/:id/feedback	technicalSkills = 6	400 rating must be 1-5	400 "technicalSkills must be a whole number from 1 to 5"	Pass
TC-36	POST /api/interviews/:id/feedback	Ratings 4/5/4 with comments	200; feedback saved	200; feedback {"t":4,"c":5,"p":4}	Pass
TC-37	GET /api/interviews	Candidate token	200; candidate's interview without recruiter feedback	200; 1 interview(s); feedback visible: false	Pass
TC-38	POST /api/offers	Offer for an application that is not "hired"	400 only for hired applications	400 "Offer letters can only be sent for hired applications"	Pass
TC-39	POST /api/offers	Status set to "hired", then valid offer	201; offer status "sent"	201; offer status "sent"	Pass
TC-40	POST /api/offers	Second offer for the same application	409 offer already exists	409 "An offer letter already exists for this application"	Pass
TC-41	GET /api/offers/candidate	Candidate token	200; one sent offer	200; 1 offer(s), position "Frontend Developer"	Pass
TC-42	GET /api/recruiter/stats	Recruiter token	200; totals reflect 2 open jobs, 1 applicant, 1 hire	200; openings 2, applicants 1, hires 1	Pass
TC-43	GET /api/admin/analytics	Candidate token, then admin token	403 for candidate; 200 with totals for admin	candidate 403; admin 200 (candidates 1, recruiters 2, jobs 2)	Pass

ID	Endpoint	Request / input	Expected result	Actual result	Status
TC-44	POST /api/candidate/saved-jobs/:job_id	Save the Backend job twice, then list	201; saved once; list contains it	201; saved list length 1	Pass
TC-45	POST /api/chatbot/message	Empty message, then a question (no GEMINI_API_KEY)	400 for empty; 200 with fallback reply and conversationId	empty 400; question 200, conversationId returned: true, mock reply: true	Pass
TC-46	DELETE /api/applications/:id	Candidate applies to Backend job, then withdraws	200 "Application withdrawn"	200 "Application withdrawn"	Pass
TC-47	DELETE /api/candidate/resume	Candidate token	200; resume removed, profile skills reset	200 "Resume removed"; latestResume now null; skills 0	Pass
TC-50	DELETE /api/candidate/resume	Check the uploaded file after TC-47 removed the resume	Stored resume file no longer exists on disk	file exists after delete: false	Pass
TC-51	POST /api/candidate/resume	Upload a 6 MB .pdf (over the 5 MB limit)	400 "File too large"	400 "File too large"	Pass
TC-48	POST /api/candidate/resume	Upload the same resume as a .docx file	201 with the same extracted fields as the PDF	201; skills [JavaScript, React, MongoDB, Git, HTML, CSS, Web Development, REST APIs]; education 1; experience 1; certifications 1	Pass
TC-49	POST /api/candidate/resume	Upload a valid PDF produced by a different generator (PyMuPDF, cross-reference stream)	201 with extracted fields	500 "Resume upload/parsing failed" (server log: pdf-parse "bad XRef entry")	Fail

Notes on the API results.

- **TC-19 and TC-51** confirm that invalid uploads are client errors: a `.txt` file returns HTTP 400 with the message "Only .pdf and .docx resumes are accepted", and a file over 5 MB returns HTTP 400 "File too large". Before the correction described in Section 10.2, the `.txt` upload returned HTTP 500 with the same message.
- **TC-47 and TC-50** confirm that removing a resume deletes both the database record and the uploaded file; before the correction, the file remained on disk.
- **TC-20 and TC-48** show that PDF and DOCX versions of the same resume produce identical extracted fields with the rule-based parser. The parsed company name of the internship was returned as "Example Labs (" because the source line contained the dates in parentheses; such small heuristic artefacts are the reason candidates can edit the extracted fields (TC-23).
- **TC-21 and TC-24** confirm the match-score calculation: three of four required skills gave 75% for the Frontend job and one of three gave 33% for the Backend job; after the candidate added Node.js in TC-23, the stored score at the time of applying was 100%.
- **TC-49** failed. A valid PDF whose internal cross-reference table is stored as a compressed stream (as written by some PDF libraries) could not be read by the bundled version of `pdf-parse`, which is based on `pdf.js` 1.10.100; the server logged "bad XRef entry" and returned HTTP 500. The same resume exported from Microsoft Word was parsed correctly (TC-20). This known issue is discussed in Section 10.3.

Table 8.4: User-interface test cases and results (executed in Chrome, 2 October 2026)

ID	Screen / action	Steps	Expected result	Actual result	Status
UI-01	Register	Open /register, enter details, keep Candidate selected, submit	Account created; home page shows the candidate overview	Redirected to /; candidate overview shown ("Good morning, Riya Test 🌞")	Pass
UI-02	Login with wrong password	Log out, open /login, enter a wrong password	Error "Invalid email or password"; user stays on login	Alert "Invalid email or password" shown; still on /login	Pass
UI-03	Protected route	While logged out, open /applications	Redirected to /login	Redirected to /login	Pass
UI-04	Resume upload	Candidate home: Upload Resume with a PDF	Profile Boost shows "AI-ready" and the file name; Resume Insights lists parsed fields	"AI-ready" and file name shown; Resume Insights: 8 skills (JavaScript, React, MongoDB, Git, HTML, CSS...), 3 experience/education/certification rows	Pass
UI-05	Edit insights	Resume Insights: Edit, add Node.js and Express, Save	Skill shown; match percentages refresh	Node.js and Express added to skills; top match scores 75%/75%/75%/67% → 100%/100%/83%/80%	Pass
UI-06	Job filters	Jobs page: work mode Remote, salary 6+ LPA, sort by Best match	Only matching jobs shown, highest match first	22 jobs shown; all Remote: true; all salary ranges reach 6+ LPA: true (lowest upper bound 7.5 LPA); match scores in descending order: true (100, 60, 33, 0, 0, 0...)	Pass
UI-07	Save job	Click the save icon on a job, open the Saved tab	Job appears under Saved	"Frontend Developer (Test)" saved and listed under the Saved tab	Pass
UI-08	Apply	Search for the recruiter's job, open it, click Apply Now; open Applications	Application listed with status Applied and match score	Listed on /applications with status Applied	Pass
UI-09	Post job	Recruiter home: Post a New Job, fill the form, submit	Success message; job listed on the recruiter's Jobs page	"Job posted successfully" shown; job listed on /recruiter/jobs	Pass
UI-10	Status change	Recruiter Applications: select the applicant, click Shortlist	Status chip and stage tracker update; candidate sees Shortlisted	Recruiter view shows Shortlisted; candidate's Applications page shows Shortlisted	Pass
UI-11	Schedule interview	Applicant: Schedule Interview, enter date, time, link and interviewer	Interview appears on both calendars; application shows Interview	Interview on 2026-10-05 10:30 listed for recruiter and candidate; candidate application shows Interview	Pass
UI-12	Feedback	Open the interview, Mark Completed, enter ratings and comments, Save Feedback	Feedback saved; not visible to the candidate	Interview marked Completed; feedback saved and shown to recruiter; visible to candidate: false	Pass
UI-13	Offer letter	Hire the applicant, Create Offer Letter, fill dates, Preview, Send; candidate opens and downloads the letter	Candidate home shows the congratulations banner; letter downloads as text	Banner "Congratulations..." shown on candidate home; letter downloaded as Offer-Letter-Nimbus-Labs-Frontend-Engineer-R5.txt	Pass
UI-14	Responsive layout	Open the site at 390 px width and open the menu	Menu button replaces navigation; cards stack in one column	At 390 px: menu button visible true, nav links hidden true, menu opens true, stat cards in one column true	Pass
UI-15	Candidate navigation link	Logged-in candidate opens the home page	Logged-in candidate sees the For Candidates link in the navigation bar	For Candidates link present for logged-in candidate: true	Pass
UI-16	Saved Jobs tile	Candidate opens the home page after saving a job	Tile on the candidate home page shows the number of saved jobs	Tile shows 1; API reports 1 saved job(s)	Pass
UI-17	Resume file picker	Inspect the resume upload controls on the candidate home page	Upload controls offer only PDF and DOCX files	accept attribute on 2 upload control(s): .pdf,.docx	Pass

Notes on the user-interface results. UI-15, UI-16 and UI-17 were added as regression tests for the corrections in Section 10.2: before those corrections, the For Candidates link was not shown to logged-in candidates, the Saved Jobs tile always displayed 0, and the upload controls also offered `.doc` files that the server rejects. The first version of the user-interface suite (UI-01 to UI-14), run against the code before the corrections, also passed all fourteen cases. Capturing the screenshots in Appendix A additionally revealed one defect that the test cases did not cover: the recruiter's applicant view always reports "No education details on file" for applications that have a resume attached, even when the parsed resume contains education entries (Section 10.3).

CHAPTER -9 DEPLOYMENT

HireHub has not yet been deployed to a public server; it is currently run on developers' machines. This chapter therefore documents, first, the exact procedure for running the system locally from the scripts and configuration files in the repository, which is the deployment that was actually used during development and testing; and second, a deployment approach for making the application publicly available. The second part is a proposal and is labelled as such: no hosting provider, domain or production URL has been set up.

9.1 Steps to Deploy

9.1.1 Local deployment (used during development)

The following steps start the complete system on a single computer. They assume Node.js and npm are installed and that a MongoDB server is running locally or that a hosted MongoDB connection string is available.

1. Obtain the source code by cloning the project's GitHub repository, and open the `project-files` folder.
2. Install the back-end dependencies: open a terminal in `job-portal-backend` and run `npm install`.
3. Create the back-end configuration: copy `.env.example` to `.env` and fill in the values described in Section 9.2 (at minimum `MONGODB_URI` and `JWT_SECRET`). To enable the AI features, also set `GEMINI_API_KEY`; it is optional.
4. Optionally load demonstration data with `npm run seed:jobs`, which creates demo recruiter accounts and about 150 job postings and can safely be run more than once.
5. Start the API with `npm run dev` (with automatic restart through nodemon) or `npm start` (plain Node.js). The console reports the MongoDB connection and "HireHub backend running on `http://localhost:8000`". Opening `http://localhost:8000/api/health` should return `{"status": "ok"}`.
6. Install the front-end dependencies: open a second terminal in `job-portal-frontend` and run `npm install`.
7. If the API is not at the default address, create a `.env` file in `job-portal-frontend` containing `VITE_API_URL` (Section 9.2).
8. Start the front end with `npm run dev` and open the address printed by Vite, normally `http://localhost:5173`.

For a production-style local check, the front end can be compiled with `npm run build`, which writes static files to the `dist` folder, and served with `npm run preview`.

9.1.2 Proposed public deployment

Proposed. The steps below describe how the existing code could be deployed publicly without changes to its source, because both applications are configured entirely through environment variables. Provider names are given only as examples of suitable services.

1. **Database:** create a managed MongoDB cluster (for example MongoDB Atlas), create a database user with a strong password, restrict network access to the API host, and note the connection string.
2. **API:** deploy `job-portal-backend` to a Node.js hosting service (for example Render or Railway) or a virtual machine. Set the build command to `npm install` and the start command to `npm start`, and set the environment variables in the host's settings rather than in a committed file.
3. **Resume storage:** attach a persistent disk to the API service for the `uploads` folder, because the current code stores uploaded resumes on the local file system and many hosting services discard local files when a service restarts. A more scalable alternative is to change the upload handling to object storage.
4. **Front end:** build `job-portal-frontend` on a static hosting service (for example Vercel or Netlify) with the build command `npm run build` and output folder `dist`, and set `VITE_API_URL` to the public API address so that it is embedded in the build. Configure the host to serve `index.html` for all paths, so that React Router can handle routes such as `/jobs/123` on a page reload.
5. **Connect the tiers:** set the API's `CORS_ORIGIN` to the public front-end address, then verify `/api/health`, registration, login, resume upload and the hiring flow on the deployed system.
6. **Optional seeding:** run `npm run seed:jobs` once against the production database only if demonstration data is wanted.

9.2 Environment Configuration

All configuration is supplied through environment variables. The back end loads them from a `.env` file with `dotenv` (an `.env.example` template is committed; the real `.env` is excluded by `.gitignore`), and the front end reads variables prefixed with `VITE_` at build time. Table 9.1 lists every variable the code reads. Real values, in particular secrets, must never be committed to the repository or included in documentation.

Table 9.1: Environment variables

Variable	Application	Required	Purpose and default
PORT	Back end	No	Port the API listens on; default 8000.
MONGODB_URI	Back end	Yes	MongoDB connection string; the server refuses to start without it. The example file uses a local database named <code>hirehub</code> .
JWT_SECRET	Back end	Yes	Secret used to sign and verify tokens; must be a long random string.
JWT_EXPIRES_IN	Back end	No	Token lifetime; default 7d.
GEMINI_API_KEY	Back end	No	Google Gemini API key; when absent, resume parsing uses the rule-based parser and the chatbot returns a fallback reply.
CORS_ORIGIN	Back end	No	Comma-separated list of front-end origins allowed to call the API; default <code>http://localhost:5173</code> .
VITE_API_URL	Front end	No	Base URL of the API, embedded at build time; default <code>http://localhost:8000/api</code> .

Two further runtime settings are fixed in code rather than configured: the maximum upload size of 5 MB and the allowed resume extensions (`.pdf` and `.docx`) in `middleware/upload.js`, and the folder used to cache the OCR language data, which is a `hirehub-tesseract` folder inside the operating system's temporary directory.

9.3 Hosting of Frontend and Backend

Current hosting. Both applications are currently hosted locally during development, as summarised in Table 4.2: the React application on the Vite development server at port 5173 and the Express API at port 8000, connected to MongoDB through `MONGODB_URI`. No public URL exists for the application.

Proposed hosting. Because the front end is a static single-page application after it has been built, it is best hosted on a static hosting service or content delivery network, which serves files quickly from locations close to users and needs no server management. The back end is a long-running Node.js process that needs environment variables, outbound access to the Gemini API, a connection to MongoDB and, in the current design, persistent disk storage for resumes; it is therefore best hosted on a Node.js platform service or a virtual machine. Separating the two in this way matches the architecture in Chapter 4 and allows each tier to be scaled independently: additional API instances can be added behind a load balancer because the API is stateless, provided that resume files are moved to shared object storage.

Table 9.2: Proposed hosting plan (not yet implemented)

Tier	Proposed hosting	Key configuration
Front end (static build)	Static hosting or CDN (for example Vercel or Netlify)	<code>VITE_API_URL</code> set to the public API URL; rewrite all paths to <code>index.html</code> .
Back end (Express API)	Node.js web service (for example Render or Railway) or a virtual machine	All back-end variables from Table 9.1; persistent disk for uploads; HTTPS.
Database	Managed MongoDB cluster (for example MongoDB Atlas)	Dedicated database user; network access limited to the API host; automated backups.

Production hosting should also enable HTTPS on both tiers (most of the services mentioned provide certificates automatically), keep `JWT_SECRET` and `GEMINI_API_KEY` in the host's secret settings, and add request rate limiting to the API before it is exposed publicly, as recommended in Chapters 10 and 11.

CHAPTER -10 CHALLENGES & LIMITATIONS

This chapter records the main technical problems encountered while building HireHub, the solutions adopted for them, and the limitations and known bugs of the current version. The issues described in Section 10.1 and in Table 10.1 are those evidenced in the source code, where the solutions and the reasons for them are documented in comments. Table 10.2 lists defects that were found by reviewing the code and testing it for this report, and that have since been corrected. The limitations and known bugs in Section 10.3 are those that remain in the current version.

10.1 Issues Faced During Development

1. **Unreliable AI availability.** Resume parsing depends on an external AI model whose availability changes over time. The code records that the model originally used, `gemini-2.5-flash`, became unavailable to newly created API keys and returned HTTP 404, which would have made every resume upload fail. The project also had to work for team members who had no API key at all.
2. **Rate limits on the free tier.** The Gemini free tier allows only a small number of requests per minute (the code notes five), and returns HTTP 429 when the limit is exceeded or HTTP 503 when the service is overloaded. Uploading several resumes in quick succession, or a long resume that requires several calls, could therefore fail intermittently.
3. **Long resumes.** Sending an entire long resume in one request risked exceeding practical input sizes, while truncating it would silently lose later sections such as certifications.
4. **Scanned resumes.** Many resumes are PDFs produced by scanning or by image-based export, which contain no text layer. For these files `pdf-parse` returns little or no text, so neither parser had anything to work with.
5. **Inconsistent resume layouts.** Resumes use many different headings, bullet styles, date formats and ways of naming the same skill, which makes rule-based extraction error-prone.
6. **Keeping skills consistent across collections.** Match scores are calculated from the skills stored on the user document, while the parsed data lives in the resume document. Any update to one had to be reflected in the other, including when a candidate edits or removes their resume.
7. **Multiple resumes and orphaned files.** Without a rule, repeated uploads would accumulate many resumes per candidate, with ambiguity about which one an application should use and with old files remaining on disk.
8. **Data that belongs to other users.** Recruiters must not see or change applications, interviews or offers for jobs posted by other recruiters, and candidates must not see recruiters' interview feedback. Route-level role checks alone could not express these rules.
9. **Role value casing.** The API stores and returns roles in lower case (for example `recruiter`), whereas parts of the front end were written with capitalised role names, which caused role checks to fail where the two were compared directly.

10.2 Solutions Applied

Table 10.1: Issues and the solutions applied

Issue	Solution in the code
Unreliable AI availability	The model name is a single constant in <code>aiResumeParser.js</code> , changed to a currently available model. Every failure path falls back to the built-in rule-based parser, and when no key is configured the rule-based parser is used from the start, so uploads never fail because of the AI service.
Rate limits	<code>extractChunk</code> retries on HTTP 429 and 503 up to four attempts, waiting for the retry delay suggested by Gemini or using exponential back-off capped at 30 seconds. A per-day quota error is recognised and not retried, so the user is not kept waiting for a quota that will not recover.
Long resumes	Text longer than 15,000 characters is split into chunks of about 12,000 characters at paragraph, line or sentence boundaries, the chunks are processed sequentially to respect rate limits, and the results are merged with duplicates removed.
Scanned resumes	If a PDF yields fewer than 100 characters, its first five pages are rendered to images and recognised with Tesseract OCR; the longer of the two texts is used. The OCR libraries are loaded only when needed.
Inconsistent layouts	The rule-based parser recognises many heading variants, bullet characters, date-range formats and skill aliases. Because no parser is perfect, candidates can review and correct every extracted field in the Resume Insights editor.
Consistency across collections	Uploading, editing and deleting a resume all update <code>User.skills</code> (and experience years) in the same request handler, so match scores always use the current skills.
Multiple resumes	Each upload deletes the candidate's previous resume documents and their files before saving the new one, so a candidate has exactly one current resume.
Data ownership	Ownership checks were added in the controllers (comparing <code>job.postedBy</code> with the requesting user), candidates' interview queries exclude the <code>feedback</code> field, and unique indexes prevent duplicate applications and offers.
Role casing	Shared helpers such as <code>isCandidateRole</code> and <code>ProtectedRoute</code> compare roles in lower case; the last remaining case-sensitive comparison, in the navigation bar, was corrected as listed in Table 10.2.

The review of the code carried out while preparing this report, together with the tests in Chapter 8, identified six further defects. They were corrected in the project code as summarised in Table 10.2, and each correction was verified by the test cases listed.

Table 10.2: Defects corrected after the code review

Defect found	Correction (file and function)	Verified by
The For Candidates link was never shown to logged-in candidates, because the navigation bar compared the stored role with "Candidate" while the API stores "candidate".	<code>job-portal-frontend/src/App.jsx</code> , <code>Navbar</code> : the candidate check now compares the role in lower case, like the recruiter check.	UI-15
Uploading a file other than PDF or DOCX returned HTTP 500, and a file over 5 MB was also reported as a server error.	<code>job-portal-backend/middleware/upload.js</code> , <code>fileFilter</code> : the rejection error now carries status 400. <code>job-portal-backend/server.js</code> , error handler: Multer errors (such as the size limit) are returned with status 400.	TC-19, TC-51
Removing a resume deleted the database record but left the uploaded file on disk.	<code>job-portal-backend/controllers/candidateController.js</code> , <code>deleteResume</code> : the stored file is now deleted as well (a missing file is logged, not treated as an error).	TC-47, TC-50

Defect found	Correction (file and function)	Verified by
The resume upload controls offered .doc files, which the server always rejects.	job-portal-frontend/src/App.jsx: the accept attribute of the five resume file inputs (in CandidateHome and CandidateProfile) is now .pdf, .docx.	UI-17
The Saved Jobs tile on the candidate home page always showed 0 with the note "Coming soon".	job-portal-frontend/src/App.jsx, CandidateHome: the saved-jobs count is loaded with getSavedJobs and shown in the tile.	UI-16
The README and .env.example described an Anthropic API key, a Postman collection that does not exist, an incorrect setup folder and an outdated description of resume parsing.	job-portal-backend/.env.example now lists GEMINI_API_KEY; job-portal-backend/README.md was corrected accordingly.	Document review

10.3 Current Limitations or Known Bugs

Known bugs

- **Some valid PDFs cannot be parsed.** The bundled pdf-parse 1.1.1 uses pdf.js 1.10.100, which fails on PDFs whose cross-reference table is stored as a compressed stream. Such uploads return HTTP 500 "Resume upload/parsing failed" (test case TC-49). Common exports, such as PDFs saved from Microsoft Word, are parsed correctly (TC-20).
- **Education is missing from the recruiter's applicant view.** The page passes the application's own resume reference to ResumeSummary in preference to the candidate's full latest resume. Because GET /api/applications populates only the file name, path and date of that resume, the Education section reports "No education details on file" for every application that has a resume attached, even when education was extracted. The skills shown are correct because they fall back to the skills stored on the user record. This was found while capturing Figure 14.7.
- **The login form requires an unused field.** The login page asks for a "Name / Username" and will not submit without it, although the API authenticates by e-mail and password only and ignores the name.

Limitations

- **Matching is exact skill overlap only.** The score counts exact (case-insensitive) matches between skill names, so related skills (for example "React" and "React.js") are treated as different unless the parser normalises them, and experience, education and location are not considered in the score.
- **Administrator dashboard not connected.** The /admin page displays fixed sample figures; the working analytics endpoint is not yet used by the interface.
- **No chatbot interface.** The AI assistant is available only through the API.
- **Security hardening is incomplete.** Any visitor can register as an administrator, because the registration form offers an Admin option and the registration endpoint accepts the admin role; there is no rate limiting or account lock-out on login; tokens cannot be revoked before they expire; and the token is stored in localStorage, which is readable by any script running on the page.

- **No notifications.** Status changes, interviews and offers are visible only inside the application; no e-mails are sent, and the offer "resend" action only records a new time.
- **Local file storage.** Resumes are stored on the API server's disk, which complicates horizontal scaling and some hosting platforms (Chapter 9).
- **Scale.** Job lists and applicant lists are not paginated; the jobs page loads all open jobs and filters them in the browser, and match scoring computes a score for every open job on each request.
- **Heuristic parsing.** Without Gemini, extraction relies on patterns and a skill dictionary; small artefacts occur (for example a trailing parenthesis in a company name in TC-20).
- **No automated tests in the repository** and no continuous-integration pipeline; the test run in Chapter 8 was carried out with a separate script.
- **Unused dependencies and bundle size.** The Three.js packages are installed but not used, and the production JavaScript bundle is about 506 kB, above Vite's recommended 500 kB chunk size.

CHAPTER -11 FUTURE ENHANCEMENTS

HireHub already implements the complete path from resume upload to offer letter, but several improvements would make it more accurate, more secure and ready for public use. The enhancements below follow directly from the limitations identified in Chapter 10 and from features whose back-end support already exists.

11.1 Planned Features

1. **Semantic job matching.** Replace or complement the exact skill-overlap score with a similarity measure based on text embeddings of the resume and the job description, so that related skills and equivalent wording are recognised. The score could also weight required experience, education and location, and explain which skills are missing for each job.
2. **Chatbot interface grounded in live data.** Add a chat panel to the candidate and recruiter dashboards that uses the existing `/api/chatbot/message` endpoint, and give the assistant read-only access to the user's own data (for example, "What is the status of my application to this job?" or "Which applicants for this role have React experience?") through controlled tool functions.
3. **Working administrator dashboard.** Connect the `/admin` page to `/api/admin/analytics` and add management functions such as viewing and deactivating users and closing inappropriate job postings.
4. **Notifications.** Send e-mail (and optionally in-app) notifications when an application's status changes, when an interview is scheduled, rescheduled or cancelled, and when an offer letter is sent, so that the offer "resend" action actually delivers the letter.
5. **Offer acceptance.** Let candidates accept or decline an offer in the application, recording the decision and notifying the recruiter.
6. **Candidate search for recruiters.** Allow recruiters to search all candidates who have opted in by skill, experience and location, using the structured resume data.
7. **Downloadable PDF offer letters** generated on the server with the company's details, instead of the current text file.
8. **Multiple resume versions.** Allow candidates to keep several resumes (for example, one per type of role) and choose which one to submit with each application.

11.2 Possible Integrations or Optimisations

- **Security hardening:** restrict the `admin` role to accounts created by an existing administrator, add rate limiting to authentication and upload routes, store tokens in HTTP-only cookies or add short-lived access tokens with refresh tokens, and add request validation middleware for all bodies.
- **Robust PDF processing:** upgrade the PDF extraction library to a current version of `pdf.js` (or use a maintained alternative) to fix the failure on PDFs with compressed cross-reference streams (TC-49).
- **Cloud storage for resumes:** store uploaded files in object storage (for example Amazon S3 or Google Cloud Storage) with signed download links, which also allows the API to scale to multiple instances.

- **Pagination and indexing:** paginate job, application and applicant lists on the server, move search and filtering to MongoDB queries, and add indexes on frequently filtered fields such as `Job.status`, `Job.postedBy` and `Application.candidate`.
- **Caching match scores:** compute and store scores when a resume or job changes rather than on every request, which becomes important as the number of jobs grows.
- **Automated testing and CI:** add the API test suite from Chapter 8 to the repository with a test runner, add front-end tests for the main user journeys, and run both automatically on every push with a continuous-integration service.
- **Deployment:** implement the hosting plan proposed in Chapter 9, with HTTPS, monitoring through the `/api/health` endpoint and regular database backups.
- **Front-end performance:** remove the unused Three.js dependencies and split the application into lazily loaded route bundles to reduce the initial download.
- **Calendar and meeting integration:** export interviews to calendar applications (iCalendar files) and generate meeting links automatically.

CHAPTER -12 CONCLUSION

12.1 Summary of the Project

HireHub is a role-based job portal that turns resumes into structured data and uses that data to match candidates with jobs. Candidates upload a PDF or DOCX resume once; the system extracts its text, recovers text from scanned documents with OCR, and structures it into skills, education, experience and certifications using the Google Gemini API or, when the AI service is unavailable, a rule-based parser. Every open job is scored by the percentage of its required skills that the candidate has, and the scores drive personalised recommendations for candidates and are stored on each application for recruiters.

Around this core, the application supports the complete hiring workflow: job posting and management, applications with a defined status pipeline, interview scheduling with calendars and structured feedback, offer letters for hired candidates, saved jobs, profiles and recruiter statistics. The system is built as a React single-page application, a Node.js and Express REST API with 31 route handlers, and a MongoDB database of seven collections modelled with Mongoose. Access is protected by JSON Web Tokens, bcrypt password hashing and role- and ownership-based authorisation.

12.2 What Was Achieved

The objectives set out in Section 1.4 were met as follows:

Table 12.1: Objectives and outcomes

Objective	Outcome
Automate resume parsing	Achieved. PDF and DOCX resumes are parsed automatically, scanned PDFs are handled with OCR, and the AI pipeline falls back to a rule-based parser, so uploads work with or without an API key. Candidates can correct the results. One limitation remains for PDFs with compressed cross-reference streams (TC-49).
Score every job by skill match	Achieved. Match scores are computed for all open jobs, power recommendations and sorting, and are stored on each application (verified in TC-21 and TC-24).
Support hiring from application to offer	Achieved. Applications, the status pipeline, interviews with feedback and offer letters are implemented end to end in both the API and the user interface (TC-24 to TC-41).
Enforce secure, role-based access	Achieved for role and ownership checks (TC-10 to TC-12, TC-16, TC-26, TC-28, TC-37, TC-43). Further hardening, such as restricting administrator registration and rate limiting, is planned (Chapter 11).
Design a sound document database	Achieved. Seven collections with validation, enumerations, references, embedded sub-documents and unique indexes protect data integrity (Chapter 5; TC-25 and TC-40).
Deliver a responsive, usable interface	Achieved for candidates and recruiters, with role-aware home pages and responsive layouts; the administrator page and chatbot interface remain to be connected.

In measurable terms, 51 API test cases covering every route group and 17 user-interface test cases executed in Chrome were run on the final code; 67 of the 68 passed, the front end builds without errors, and six defects found during the review were corrected and verified (Table 10.2). The project therefore delivers a working, tested

foundation for an AI-assisted recruitment platform, together with a clear list of the improvements needed before public deployment.

12.3 Skills Learned During Development

Building HireHub developed skills across database design, back-end engineering and front-end development:

- **Database design:** modelling a real workflow as related MongoDB collections, choosing between embedding and referencing, and protecting integrity with schema validation, enumerations and unique compound indexes; using `populate` and the aggregation framework.
- **REST API design:** organising an Express application into routes, middleware, controllers, models and utilities; designing consistent endpoints and status codes; and handling multipart file uploads.
- **Security:** implementing stateless authentication with JWT, hashing passwords with `bcrypt`, and separating role-based route protection from ownership-based authorisation in controllers.
- **Integrating external AI services:** prompting a large language model for structured JSON output with a response schema, chunking long inputs, handling rate limits with retries and back-off, and designing a fallback so the system degrades gracefully.
- **Document processing:** extracting text from PDF and DOCX files and applying OCR to scanned documents, and writing heuristic text parsers with regular expressions.
- **Front-end development:** building a multi-page React application with React Router, protected routes, component state and effects, parallel data loading, client-side filtering and sorting, and responsive CSS.
- **Testing:** writing an ordered integration test scenario, isolating test data with an in-memory database, and analysing failures from server logs.
- **Collaboration and tooling:** using Git and GitHub, environment-based configuration that keeps secrets out of the repository, npm scripts and Vite.

CHAPTER -13 REFERENCES

The following official documentation, standards, papers and books were used for the design, implementation and testing of HireHub. Online resources were consulted during development in 2026.

1. Meta Platforms, Inc., "React Documentation." [Online]. Available: <https://react.dev/>
2. Vite Contributors, "Vite Guide." [Online]. Available: <https://vite.dev/guide/>
3. Remix Software, Inc., "React Router Documentation." [Online]. Available: <https://reactrouter.com/>
4. OpenJS Foundation, "Node.js Documentation." [Online]. Available: <https://nodejs.org/docs/latest/api/>
5. OpenJS Foundation, "Express 4.x API Reference." [Online]. Available: <https://expressjs.com/en/4x/api.html>
6. MongoDB, Inc., "The MongoDB Manual." [Online]. Available: <https://www.mongodb.com/docs/manual/>
7. Mongoose Contributors, "Mongoose Documentation." [Online]. Available: <https://mongoosejs.com/docs/>
8. Google, "Gemini API Documentation." [Online]. Available: <https://ai.google.dev/gemini-api/docs>
9. M. Jones, J. Bradley and N. Sakimura, "JSON Web Token (JWT)," RFC 7519, Internet Engineering Task Force, May 2015. [Online]. Available: <https://www.rfc-editor.org/rfc/rfc7519>
10. N. Provos and D. Mazières, "A Future-Adaptable Password Scheme," in Proc. USENIX Annual Technical Conference, FREENIX Track, Monterey, CA, USA, 1999.
11. R. T. Fielding, "Architectural Styles and the Design of Network-based Software Architectures," Ph.D. dissertation, University of California, Irvine, 2000.
12. P. P. Chen, "The Entity-Relationship Model—Toward a Unified View of Data," ACM Transactions on Database Systems, vol. 1, no. 1, pp. 9–36, 1976.
13. R. Smith, "An Overview of the Tesseract OCR Engine," in Proc. 9th International Conference on Document Analysis and Recognition (ICDAR), 2007, pp. 629–633.
14. C. de Ruijt and S. Bhulai, "Job Recommender Systems: A Review," arXiv preprint arXiv:2111.13576, 2021.
15. S. Bradshaw, E. Brazil and K. Chodorow, MongoDB: The Definitive Guide, 3rd ed. Sebastopol, CA, USA: O'Reilly Media, 2019.
16. S. Chacon and B. Straub, Pro Git, 2nd ed. Berkeley, CA, USA: Apress, 2014. [Online]. Available: <https://git-scm.com/book>
17. Express.js, "Multer – Node.js middleware for handling multipart/form-data." [Online]. Available: <https://github.com/expressjs/multer>
18. Project Naptha, "Tesseract.js – Pure JavaScript OCR." [Online]. Available: <https://github.com/naptha/tesseract.js>
19. M. Williamson, "Mammoth .docx to HTML converter (mammoth.js)." [Online]. Available: <https://github.com/mwilliamson/mammoth.js>
20. Mozilla, "PDF.js." [Online]. Available: <https://mozilla.github.io/pdf.js/>

21. Typegoose, "mongodb-memory-server." [Online]. Available: <https://github.com/typegoose/mongodb-memory-server>
22. Motion Division, "Motion (formerly Framar Motion) Documentation." [Online]. Available: <https://motion.dev/docs>
23. MDN Web Docs, "Fetch API." [Online]. Available: https://developer.mozilla.org/en-US/docs/Web/API/Fetch_API

CHAPTER -14 APPENDICES

Appendix A – Screenshots of the App

Figures 14.1 and 14.2 show the role-aware home page of the running application for a logged-in candidate and a logged-in recruiter.

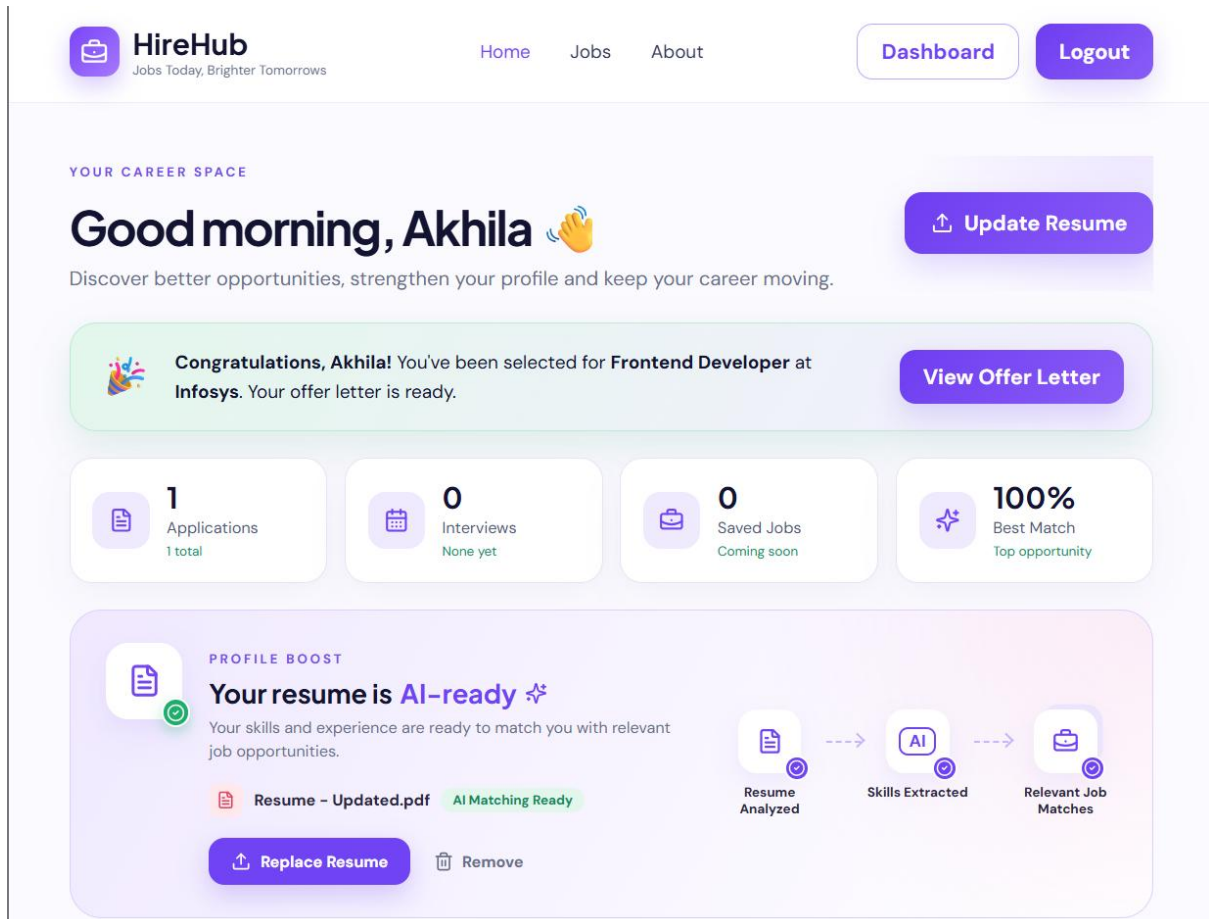


Figure 14.1: Candidate home page – career space, offer banner, statistics and Profile Boost

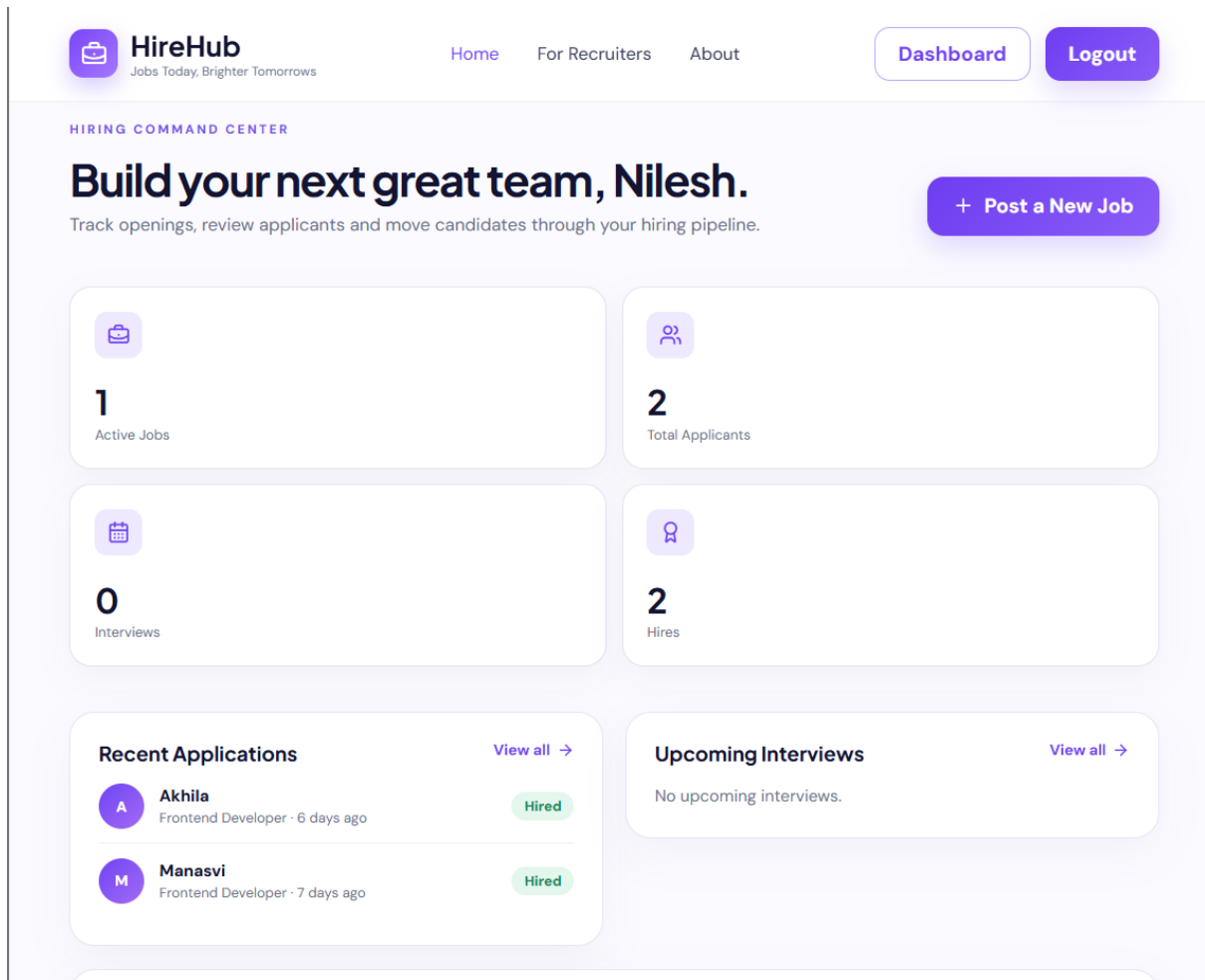


Figure 14.2: Recruiter home page – hiring command center, KPIs and recent applications

Figures 14.3 to 14.9 show the remaining screens. They were captured from a separate test instance of the application loaded with the project's seeded demonstration jobs and a small demonstration scenario; all names, e-mail addresses and companies in them are fictional.

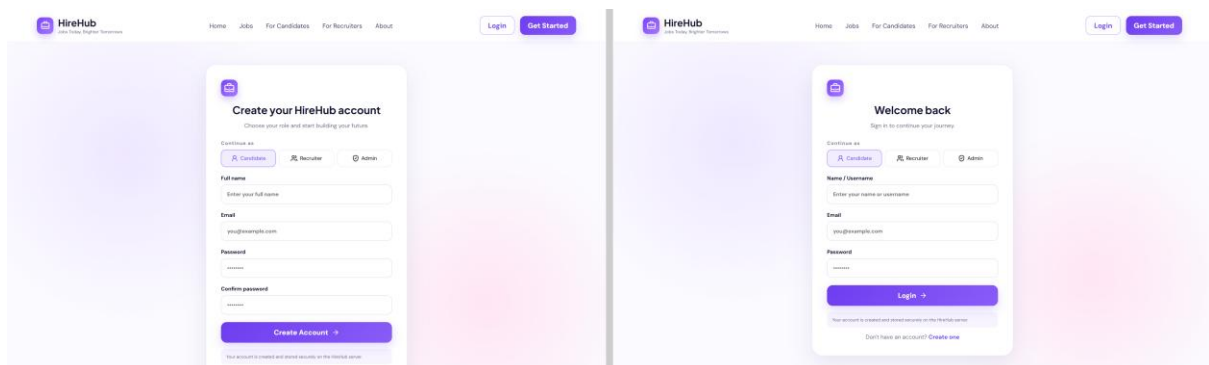


Figure 14.3: Registration (left) and login (right) pages

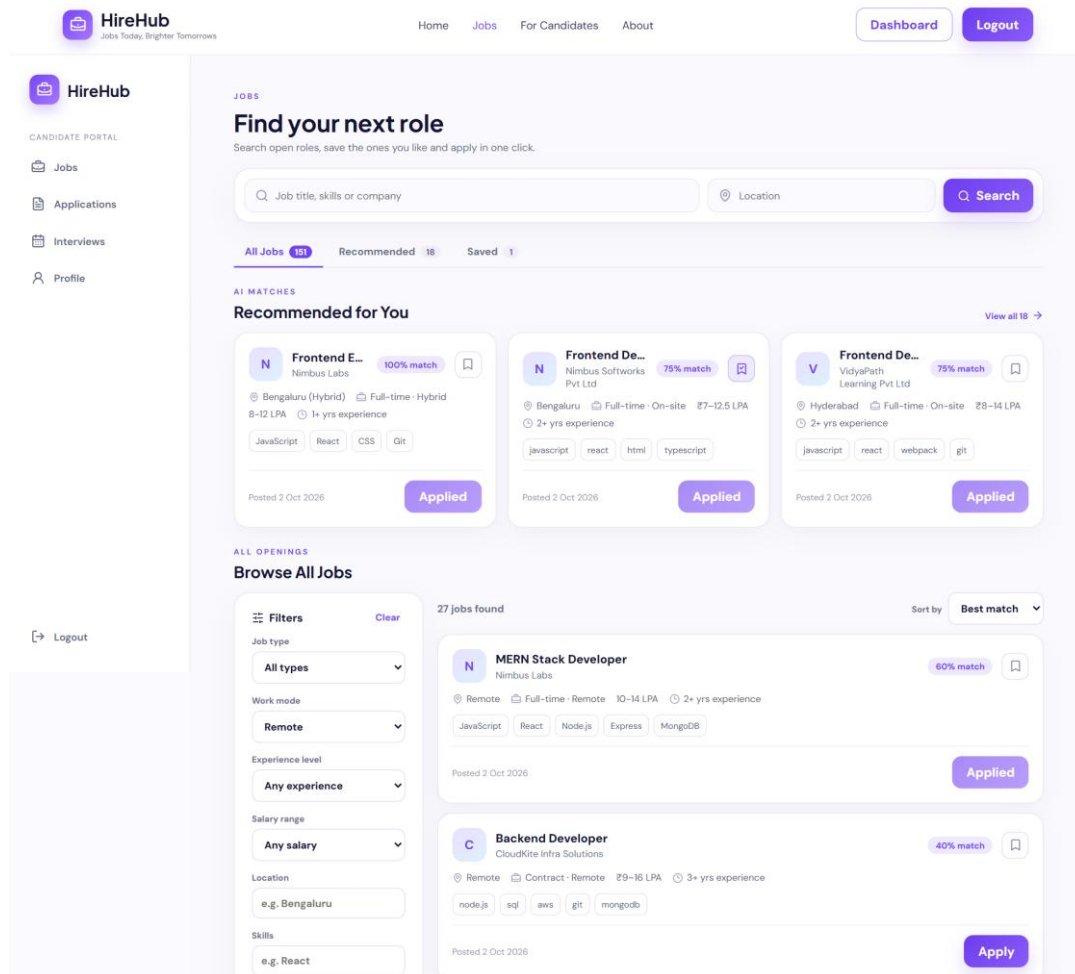


Figure 14.4: Candidate Jobs page – tabs, AI recommendations, filters and best-match sorting

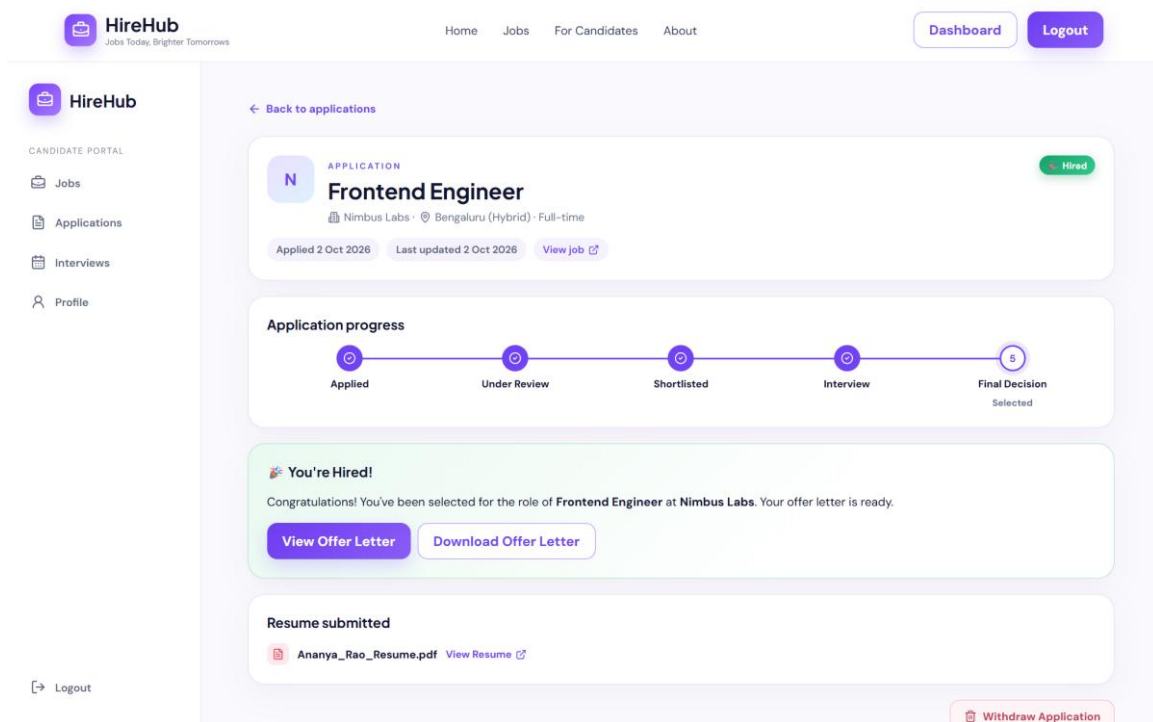


Figure 14.5: Candidate application detail – stage tracker and offer letter for a hired application

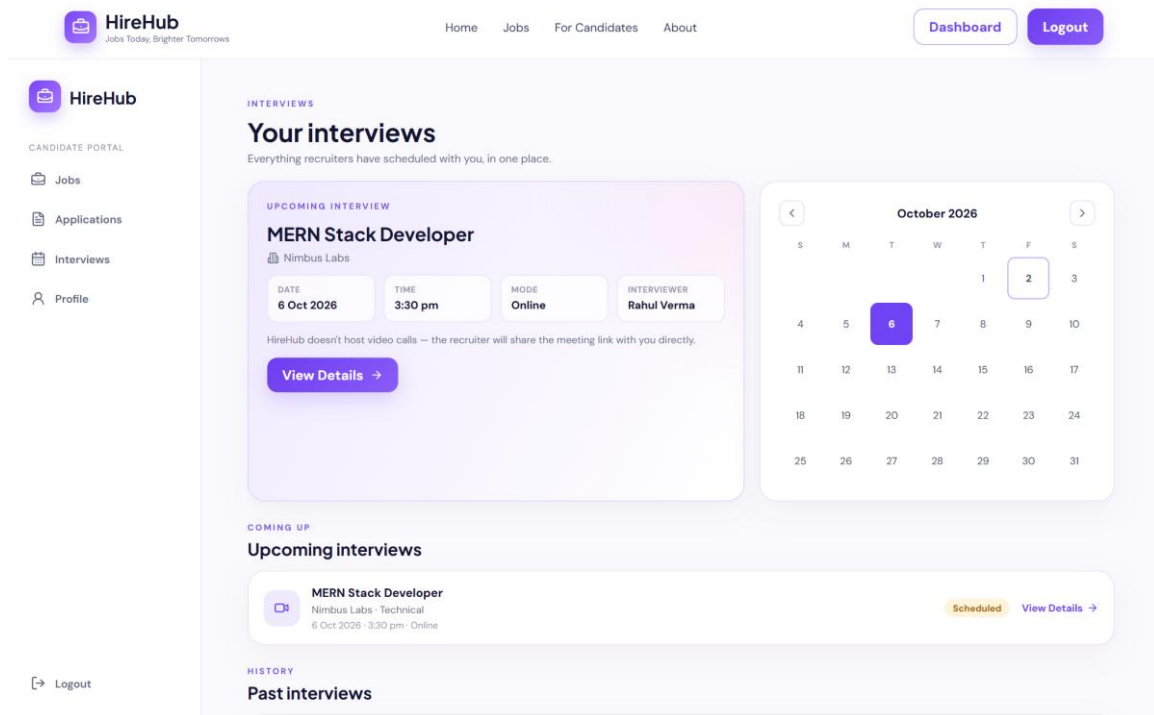


Figure 14.6: Candidate Interviews page – upcoming interview, calendar and history

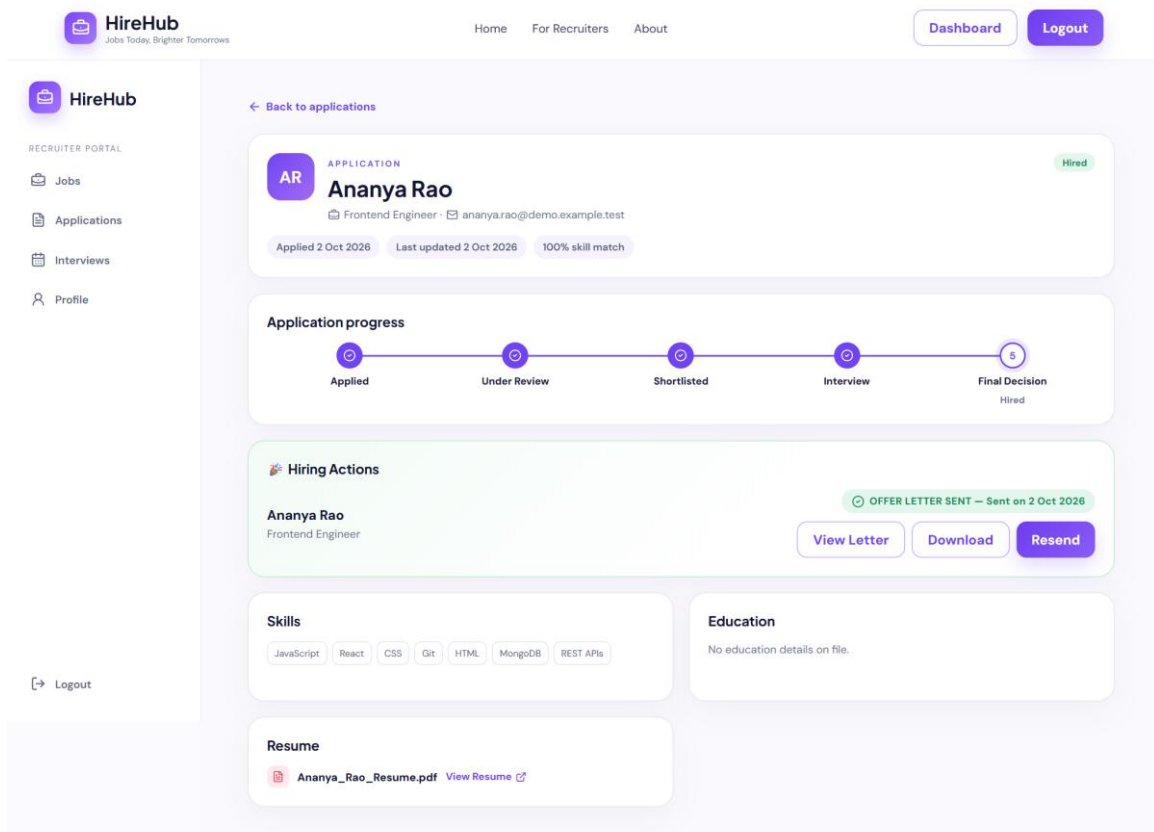


Figure 14.7: Recruiter applicant view – match score, stage tracker, Hiring Actions and parsed resume

The screenshot shows the HireHub Recruiter Portal with a sidebar containing 'Jobs', 'Applications', 'Interviews', and 'Profile'. The main content area is titled 'Interviews' and includes a 'Schedule Interview' modal form. The form fields are as follows:

Candidate & Job	
Karan Iyer – MERN Stack Developer (Interview)	
Date dd-mm-yyyy	Time --:--
Interview type Technical	Mode Online
Meeting link (optional) e.g. https://meet.google.com/abc-defg-hij	
Interviewer (optional) e.g. Priya Sharma	
Notes for the candidate (optional) What the round covers, what to bring...	

Buttons at the bottom of the form: Cancel, Schedule Interview.

Figure 14.8: Recruiter Interviews page – scheduling form

The screenshot shows the 'Interview detail' page for Ananya Rao. The page includes a 'Back to interviews' link and a summary of the interview details:

INTERVIEW					
Ananya Rao					
Frontend Engineer · ananya.rao@demo.example.test					
STATUS Completed	DATE 30 Sept 2026	TIME 11:00 am	TYPE Technical	MODE Online	INTERVIEWER Priya Sharma
MEETING LINK https://meet.example.test/fe-round					

Below the summary, there is a section for 'Notes shared with the candidate' (Live coding with React components) and an 'Interview Feedback' section. The feedback section includes a table of scores and a note:

Interview Feedback			
Internal to your team – candidates don't see this.			
TECHNICAL SKILLS 5 / 5	COMMUNICATION 4 / 5	PROBLEM SOLVING 4 / 5	RECOMMENDATION Strong Hire

Feedback note: Strong React fundamentals and clear explanations.

Figure 14.9: Recruiter interview detail – completed interview with feedback

Appendix B – Sample Code Snippets

The following excerpts are taken unchanged from the project source code (some long files are shown in part).

Listing 14.1: job-portal-backend/models/Application.js (schema and index)

```
const applicationSchema = new mongoose.Schema(
  {
    job: { type: mongoose.Schema.Types.ObjectId, ref: "Job", required: true },
    candidate: { type: mongoose.Schema.Types.ObjectId, ref: "User", required: true },
    resume: { type: mongoose.Schema.Types.ObjectId, ref: "Resume" },
    matchScore: { type: Number, default: 0 }, // 0-100
    status: {
      type: String,
      enum: ["applied", "under_review", "shortlisted", "interview", "rejected", "hired",
"withdrawn"],
      default: "applied",
    },
  },
  { timestamps: true }
);

// A candidate can only apply once per job
applicationSchema.index({ job: 1, candidate: 1 }, { unique: true });
```

Listing 14.2: job-portal-backend/controllers/applicationController.js (applyToJob)

```
export async function applyToJob(req, res) {
  try {
    if (req.user.role !== "candidate") {
      return res.status(403).json({ message: "Only candidates can apply to jobs" });
    }

    const { job_id } = req.params;
    const job = await Job.findById(job_id);
    if (!job) return res.status(404).json({ message: "Job not found" });

    const existing = await Application.findOne({ job: job_id, candidate: req.user.id });
    if (existing) {
      return res.status(409).json({ message: "You have already applied to this job" });
    }

    const user = await User.findById(req.user.id);
    const latestResume = await Resume.findOne({ candidate: req.user.id }).sort({ createdAt:
-1 });
    const matchScore = calculateMatchScore(user.skills, job);

    const application = await Application.create({
      job: job_id,
      candidate: req.user.id,
      resume: latestResume ? latestResume._id : undefined,
      matchScore,
    });

    return res.status(201).json({ application });
  } catch (err) {
    console.error("[applyToJob]", err);
    return res.status(500).json({ message: "Could not submit application" });
  }
}
```

Listing 14.3: job-portal-backend/middleware/upload.js (file filter and limits)

```
function fileFilter(req, file, cb) {
  const allowed = [".pdf", ".docx"];
  const ext = path.extname(file.originalname).toLowerCase();
  if (allowed.includes(ext)) {
    cb(null, true);
  } else {
    cb(new Error("Only .pdf and .docx resumes are accepted"));
  }
}

export const uploadResumeFile = multer({
  storage,
  fileFilter,
  limits: { fileSize: 5 * 1024 * 1024 }, // 5MB
});
```

Listing 14.4: job-portal-frontend/src/App.jsx (ProtectedRoute)

```
function ProtectedRoute({ children, roles }) {
  const auth = getAuth();

  if (!auth?.isAuthenticated) {
    return <Navigate to="/login" replace />;
  }

  if (roles) {
    const normalizedRoles = roles.map((r) => r.toLowerCase());
    if (!normalizedRoles.includes((auth.role || "").toLowerCase())) {
      return <Navigate to={getDashboardPath(auth.role)} replace />;
    }
  }

  return children;
}
```

Appendix C – Installation / Setup Instructions

Prerequisites: Node.js 18 or later with npm, and either a running MongoDB server or a hosted MongoDB connection string. A Google Gemini API key is optional.

Listing 14.5: Setup commands

```
# 1. Back end
cd project-files/job-portal-backend
npm install
copy .env.example .env           # macOS/Linux: cp .env.example .env
# edit .env: set MONGODB_URI, JWT_SECRET, CORS_ORIGIN and (optionally) GEMINI_API_KEY
npm run seed:jobs                # optional: demo recruiters and ~150 jobs
npm run dev                      # API on http://localhost:8000 (npm start without nodemon)

# 2. Front end (second terminal)
cd project-files/job-portal-frontend
npm install
# optional .env: VITE_API_URL=http://localhost:8000/api
npm run dev                      # open http://localhost:5173

# 3. Production build of the front end
npm run build                    # output in dist/
npm run preview                  # serve the build locally
```

To verify the installation, open <http://localhost:8000/api/health>, which should return `{"status": "ok"}`, then register a candidate and a recruiter in the browser and follow the user guide in Appendix D. The variables used in `.env` are described in Table 9.1.

Appendix D – User Manual / Guide

For candidates

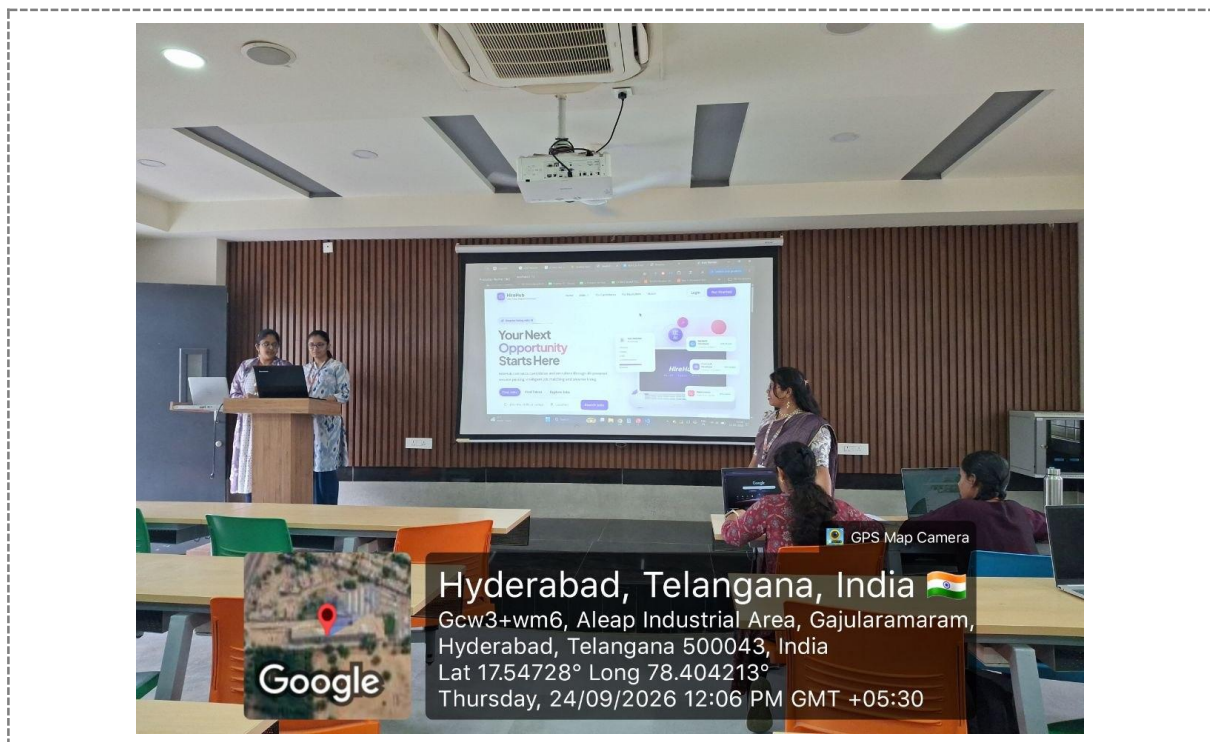
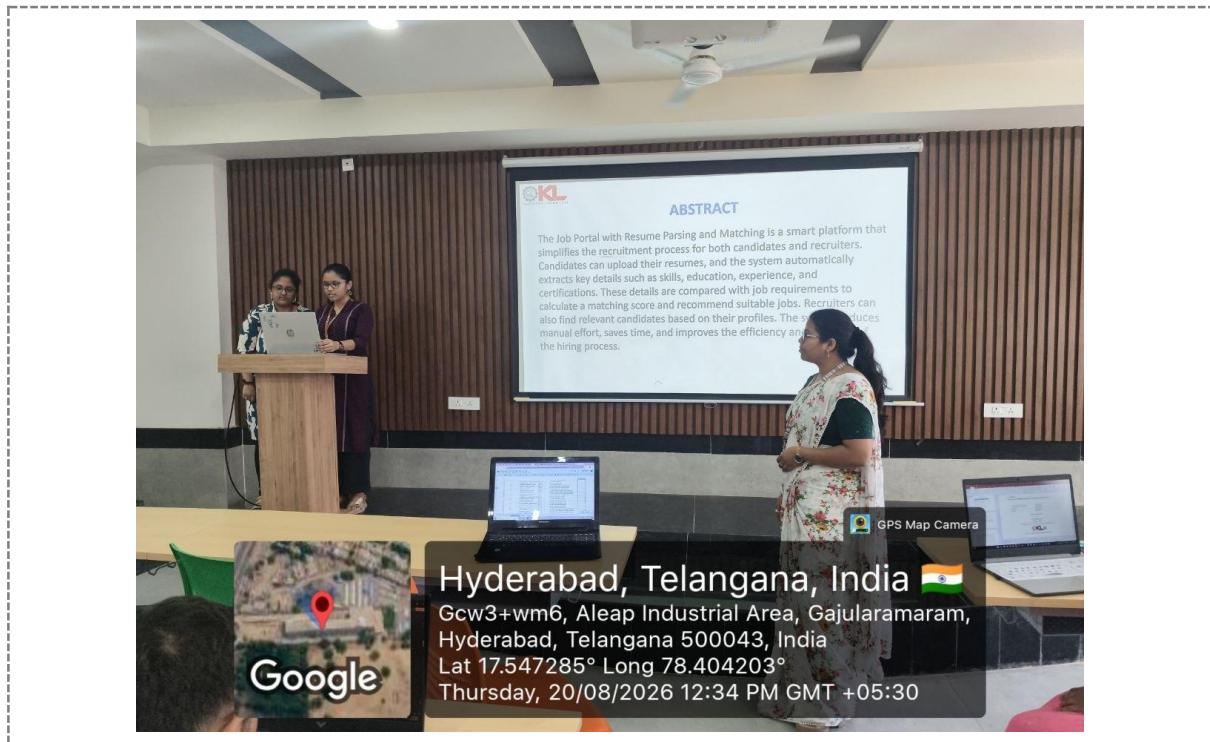
1. Open the application, choose Get Started, select Candidate and register with your name, e-mail and password.
2. On the home page, click Upload Resume and select your resume as a PDF or DOCX file of up to 5 MB. Wait for the Profile Boost card to show that your resume is AI-ready.
3. Check the Resume Insights section. Click Edit to correct or add skills, education, experience and certifications, then save.
4. Open Jobs. Use the search box and filters, switch to the Recommended tab to see jobs that match your skills, or sort by best match. Use the bookmark icon to save jobs for later (Saved tab).
5. Open a job to read the full description and your match percentage, and click Apply.
6. Follow your applications under Applications. The status chip and stage tracker show where each application stands; you can withdraw an application that is still in progress.
7. When an interview is scheduled, open Interviews to see it on the calendar with its date, time, mode, meeting link or location and notes.
8. If you are selected, a congratulations banner appears on your home page. Click View Offer Letter to read the offer and download it.
9. Complete your Profile (title, phone, location, LinkedIn, GitHub and About) to raise your profile-completion score.

For recruiters

1. Register with the Recruiter option, then open Profile and enter your company details.
2. On the home page or the Jobs page, click Post a New Job and enter the title, company, location, type, description, required skills (comma-separated), experience and salary range. Required skills are used to calculate candidates' match scores.
3. Manage your jobs on the Jobs page: edit a posting, or close it to stop receiving applications and reopen it later.
4. Open Applications to review applicants. Filter by job, stage or experience, select an applicant to see the match score, parsed resume and original resume file, and move the application through the stages Under Review, Shortlisted, Interview, Hired or Rejected.
5. Click Schedule Interview for an applicant, fill in the date, time, type, mode and meeting link or location, and save. Manage interviews from the Interviews page: reschedule, cancel or mark them completed, then record feedback ratings and a recommendation.

6. When an applicant is Hired, use Hiring Actions to create the offer letter, preview it and send it to the candidate. Sent offers can be viewed, downloaded or re-sent.
7. Use the home page to monitor active jobs, applicants, interviews and hires.

Appendix E – Geo Tag Photos with Guide



Appendix F – Review Forms with Guide Signatures

PLACEHOLDER

Scanned review form with guide signature – Review 1

PLACEHOLDER

Scanned review form with guide signature – Review 2